

윤수영 포트폴리오

1. 최종프로젝트 : 스트레스 예측을 위한 안면 이미지 데이터 분석 프로젝트
2. 과제 : OCR을 사용해서 나이브베이즈로 훈련한 금칙어 데이터 모델을 활용한 클린 게시판 만들기
3. 중간 프로젝트 : AI기반 이미지 처리 머신러닝 & 딥러닝 프로젝트

연락처 : 010-2175-3797
이메일 : ysy_itc@naver.com

목차

1. 이력서

2. 최종프로젝트 : 스트레스 예측을 위한 안면 이미지 데이터 분석 프로젝트

a. Kobert를 이용한 한국어 감정 인식 모델

b. 전처리 : KoBert를 사용한 부사 증강, 데이터 길이 중간값으로 맞추기

3. 과제 : OCR과 나이브베이즈로 훈련한 금칙어 데이터 모델이 적용된 클린 게시판 만들기

a. 금칙어 데이터 수집

b. 나이브베이즈로 데이터 훈련

4. 중간 프로젝트 : AI기반 이미지 처리 머신러닝 & 딥러닝 프로젝트

a. OCR을 사용한 신분증 인식 회원가입 기능

b. 신분증 데이터 생성

c. Google Cloud Vision을 사용하여 이미지 내 텍스트 데이터 추출

d. 전처리 : 주민등록증/면허증 인 이미지의 개인 정보 데이터 수집

이력서



윤수영 여 1997년 (만 27세)

휴대폰	010 -2175 - 3797	Email	ysy_itc@naver.com
주소	경기도 하남시 학암동		

학력

기간	학교명	전공	학점
2016.02 ~ 2020.03 (졸업)	명지전문대학	정보통신공학과 (공학사)	3.69 / 4.5
2013.03 ~ 2016.02 (졸업)	가좌고등학교		

자격증

취득년월	자격증명	발행처
2024. 04	SQLD	한국데이터산업진흥원
2022. 11	정보처리기사	한국산업인력공단

경력

재직기간	회사명	부서명	주요직무
2020.10 ~ 2023.12 (3년 3개월 , 퇴사)	컴즈 (주)	QA 사업본부 대리	SW 품질 검증 및 자동화 개발

교육

교육기간	기관명	과정명
2024.02 ~ 2024.08	한국ICT인재교육원	자바&파이썬 풀스택과정
학습내용		

프론트엔드 (Html5, Css3, Javascript, Vue.js, React 등)와 백엔드 (Java, Python, Oracle, Docker 등)를 바탕으로 풀스택 개발 과정 강의를 수강하였습니다.
추가로 Numpy, Pandas를 활용한 머신러닝/딥러닝 수업을 수강하여 OCR을 활용한 신분증/신용카드/영양성분표 글자 인식 기능, AI Hub의 데이터를 활용하여 안면 이미지 내 표정 분석을 통한 영상 내 사용자의 표정 감정 분석 기능, 한글 문장의 감정 인식 기능 등을 개발하였습니다.
* 포트폴리오 첨부
* 최종 프로젝트 Git : https://github.com/swimming123/S2chat_final

최종프로젝트

스트레스 예측을 위한 안면 이미지 데이터 분석 프로젝트

팀장 윤수영

팀원 박성호 허호준 정준영 김건우 송지미 이승희 이지영

윤수영 작업 내용

머신러닝/딥러닝

- KoBert를 이용한 감정인식 모델 구현
- 전처리 : KoBert를 사용한 부사 증강
- Google Cloud Vision과 나이브베이즈를 사용한 스팸단어 검출 모델 구현
- 전처리 : KoNLPy의 Okt를 사용한 한국어 문장 토큰화

SERVICES

- Spring Boot, Vue, Axios를 이용한 마이다이어리 감정 인식 서비스 구현
- TTS, STT 기능을 사용한 간단 심리 검사 서비스 구현

기술 스택

Front: Vue, axios, Text To Speech, High Chart
Back: SpringBoot ,Mybatis, Oracle, Python, Django

DB PL/SQL 구현

- Oracle DB를 사용한 마이 다이어리 데이터 전송 트리거 구현

배포

- Github를 이용한 형상관리
- Git Action을 이용한 자동 배포 (Django)
- AWS를 이용한 자동 배포 (Django)

최종프로젝트 | 기획의도

담당 : 윤수영

문제 상황 시뮬레이션 서비스를 기획하게 된 계기

- 젊은 연령의 사람들이 예측할 수 없는 대인 관계에 대해 어려워 하고 있으며 적극적인 치료가 되지 않는 경우가 많아
- 비대면으로 시작하는 시뮬레이션 서비스로
- 불안감 완화, 대인 관계의 이해, 정신적인 준비를 할 수 있도록 하기 위해 기획하게 됨

불안 상황, 가상현실로 훈련... VR 게임으로 정신과 질환 치료한다

이기상 헬스조선 기자 | 입력 2017.08.23 04:30

사회불안·공포증·중독에 효과
스마트폰 연결, 집에서도 가능

집에서 하는 VR 게임 치료가 사회불안장애 환자의 불안 척도 점수를 개선하는 데 효과가 있음을 입증했다. 2명의 사회불안장애 환자에게 VR 장치를 이용해 집에서 게임 치료를 실시하게 했다. 타인 앞에서 발표하는 상황 등을 간접 체험하는 게임이었다. 환자들의 사회불안척도 평균 점수는 2주 만에 66.27점에서 53.18점으로 크게 감소했다. 김재진 교수는 "환자가 평소 불안을 느끼는 상황을 VR 기기로 간접 체험하게 하면, 실제 비슷한 상황에 맞닥뜨렸을 때 불안감을 낮추는 효과가 있다"고 말했다.

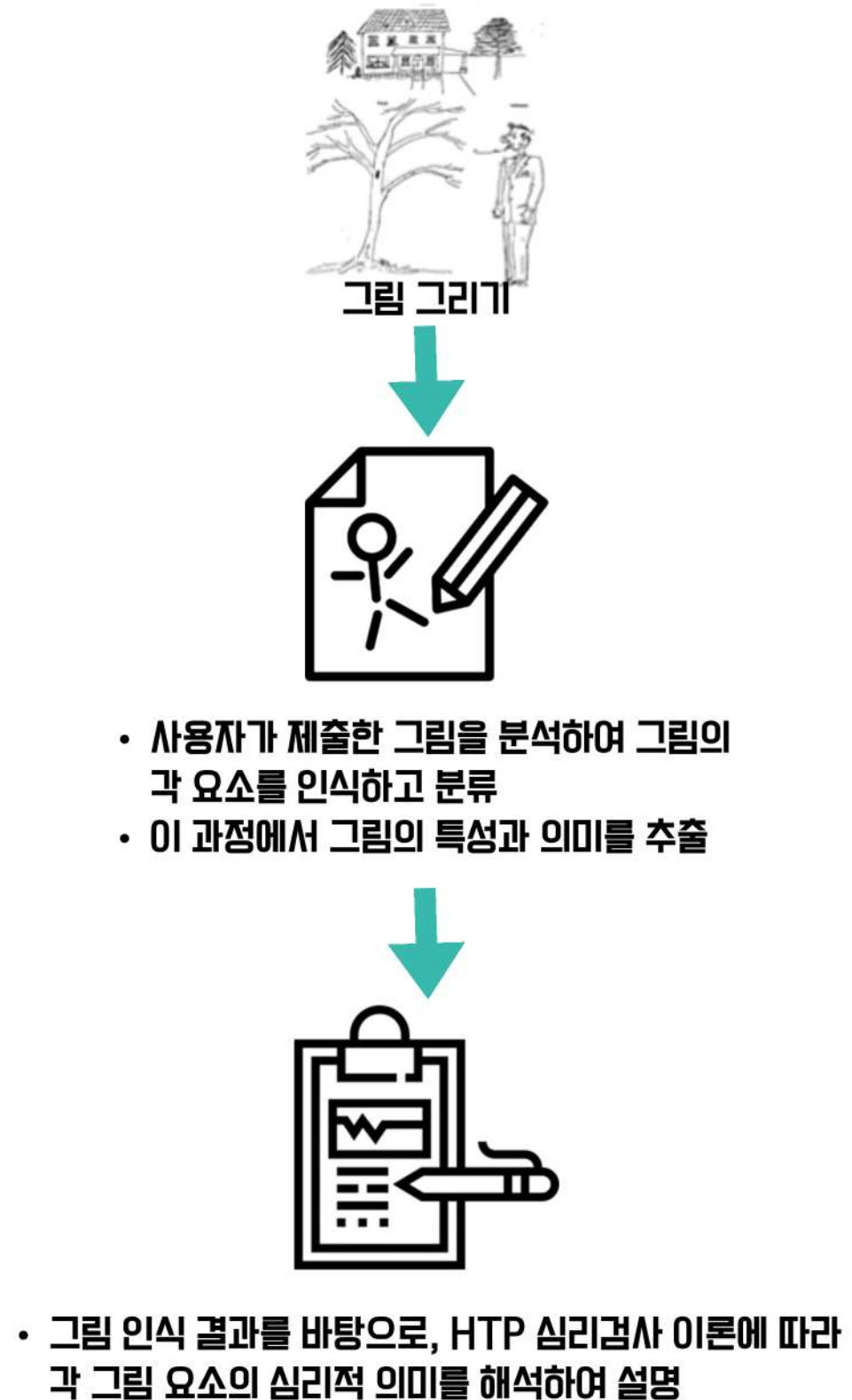
최종프로젝트 | 주요기능

담당 : 윤수영

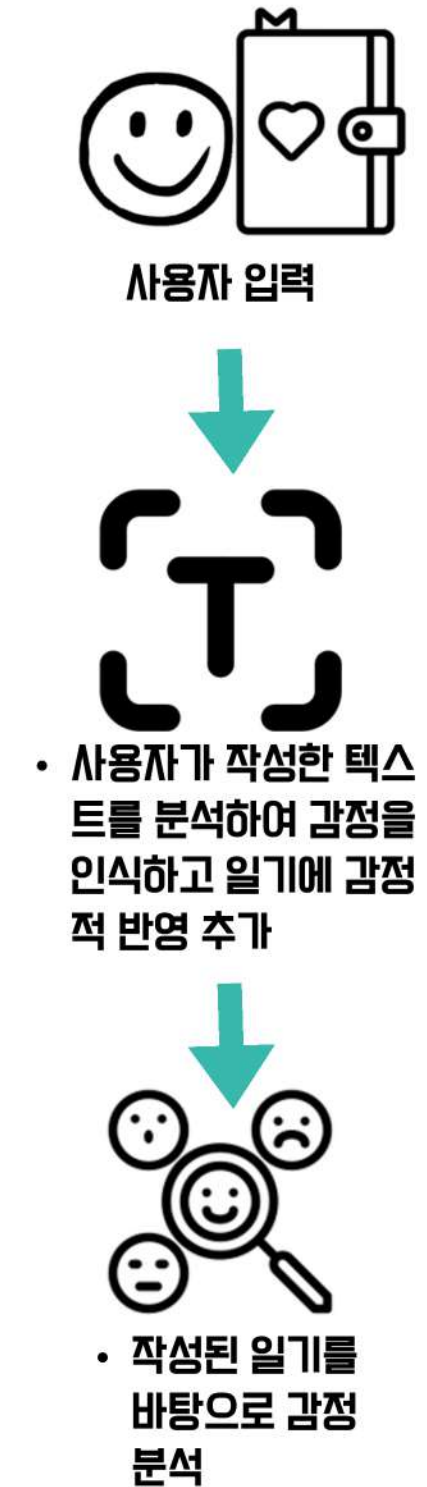
S2 시뮬레이션



HTP 검사



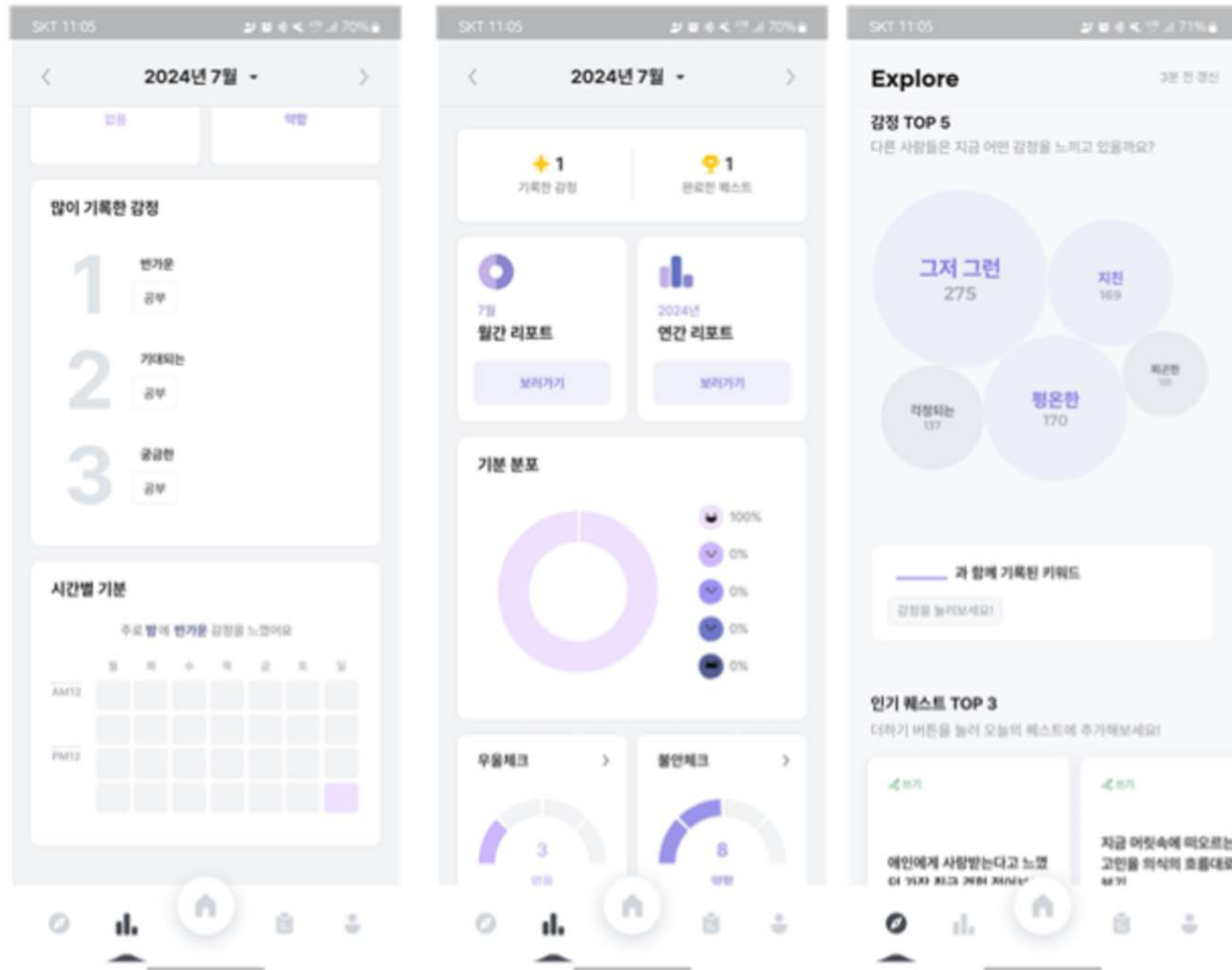
마이다이어리



최종프로젝트 | 벤치마킹

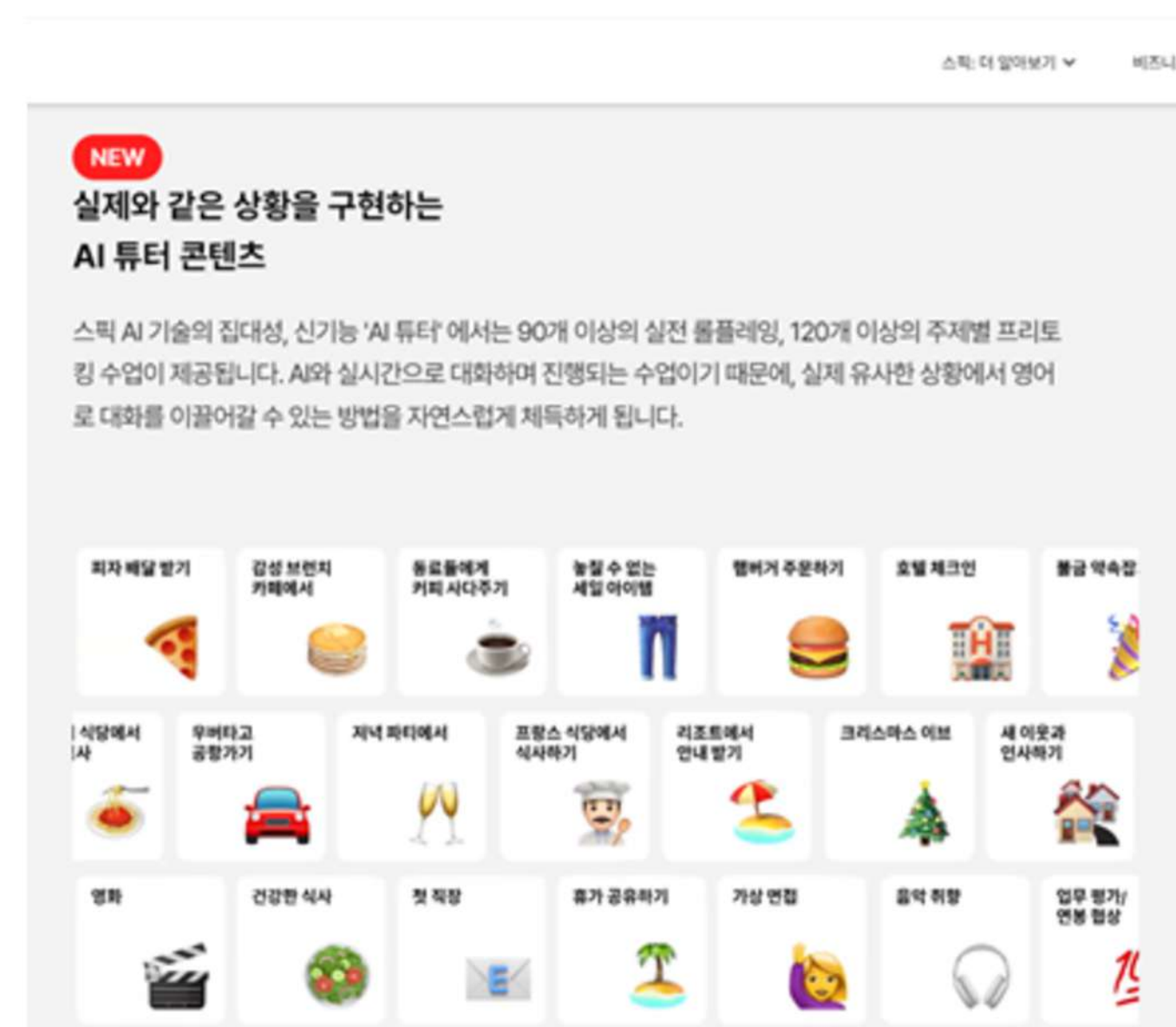
담당 : 윤수영

앱 : 무디



- ai 채팅 기능을 포함한 **고객의 데이터로 많은 통계 서비스를 제공하는 기능을 벤치마킹**
- 다양한 심리 진단 검사와 시뮬레이션 서비스를 활용하여 **고객의 정보를 수집하고**
- 캘린더 형식이 아닌 **감정 서비스를 이용한 후에 즉시 개인 보고서 형식으로 차별화**

앱 : 스픽



- 스픽의 **AI 튜터 콘텐츠를 참고하여 시뮬레이션 페이지를 벤치마킹**
- 사용자가 **시와 대화하며 실제 상황과 유사한 시뮬레이션을 경험할 수 있도록 기획**

최종프로젝트 | 개발 일정

담당 : 윤수영

SUN	MON	TUE	WED	THU	FRI	SAT
					7/19	7/20
					주제 회의	
7/21	7/22	7/23	7/24	7/25	7/26	7/27
	주제회의, 개발환경 확인, 예상일정 작성, 역할분담			템플릿 선정, UI 디자인		
7/28	7/29	7/30	7/31	8/1	8/2	8/3
	ERD 제작 및 DB 구축		1차 개발 기간 : Front - Vue, Back - Spring boot 개발			
8/4	8/5	8/6	8/7	8/8	8/9	8/10
1차 개발 기간 : Front - Vue, Back - Spring boot 개발			2차 개발 기간 : DataBase와 Spring Boot 연결, 딥러닝 모델 학습			
8/11	8/12	8/13	8/14	8/15	8/16	8/17
3차 개발 기간 : 전체 코드 머지 후 오류 수정, 딥러닝 모델 Django 적용				4차 개발 기간 : 로그인 적용, API 적용		
8/18	8/19	8/20	8/21	8/22	8/23	8/24
	최종 개발 기간 : 잔여 이슈 확인 및 수정, 기능 정리					
8/25	8/26	8/27	8/28			
	발표준비	발표	수료			

최종프로젝트 | 사용기술 · 개발도구

담당 : 윤수영

IDE | SERVER



FRONT



BACK



• 학습 방법 참고 :

- <https://velog.io/@fhflwhwl5/Python-KoBERT-7가지-감정의-다중감성분류모델-구현하기>
- <https://zzgrworkspace.tistory.com/118>

• Data 출처

- **한국인 감정인식을 위한 복합 영상**
 - <https://aihub.or.kr/aihubdata/data/view.do?currMenu=115&topMenu=100&aihubDataSe=realm&dataSetSn=82>
- **감정 분류를 위한 대화 음성 데이터셋**
 - <https://www.aihub.or.kr/aihubdata/data/view.do?currMenu=&topMenu=&dataSetSn=263&aihubDataSe=extrldata>

• IDE

- visual Code : 1.91.1
- SQL Gate : 9.19.3
- Postman : v11.6.1

• Software

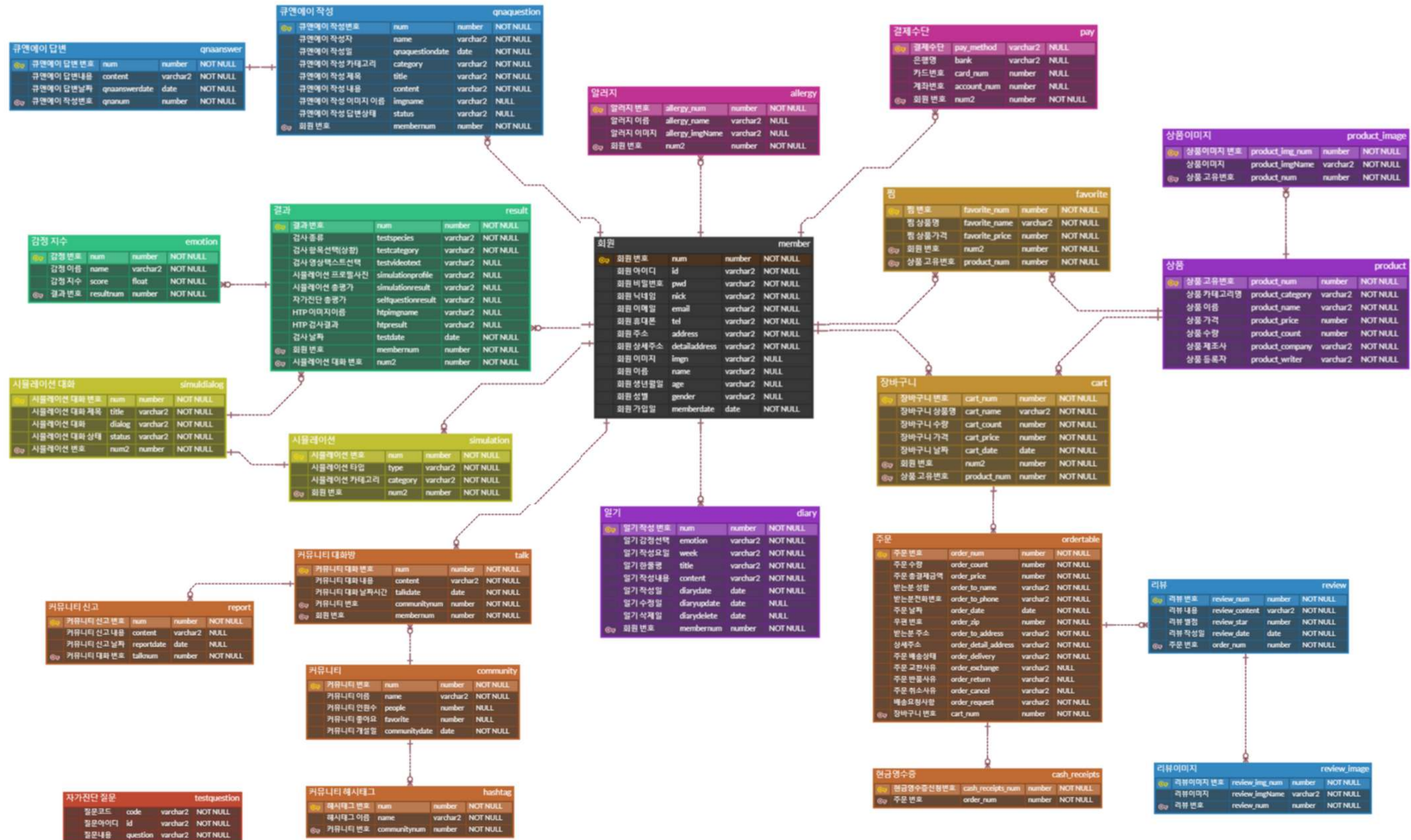
- window 11
- Ubuntu 22.04.4LTS
- java : "17.0.10" 2024-01-16 LTS
- python : 3.12.3
- node : v20.12.2
- Spring boot : 3.2.6
- Oracle(11g)
- Oracle VM VirtualBox : 6.1
- ubuntu : 22.04.4LTS
- pycharm : 2024.1.3
- python : 3.10.14
- dJango : 5.0.4
- google colab
- Python 3.10.12

• 사용한 모듈

- sklearn : naive_bayes : GaussianNB
- sklearn : feature_extraction.text : CountVectorizer
- google-cloud-vision
- core : 1.31.5
- google-cloud-vision : 2.11.1
- google-cloud-storage

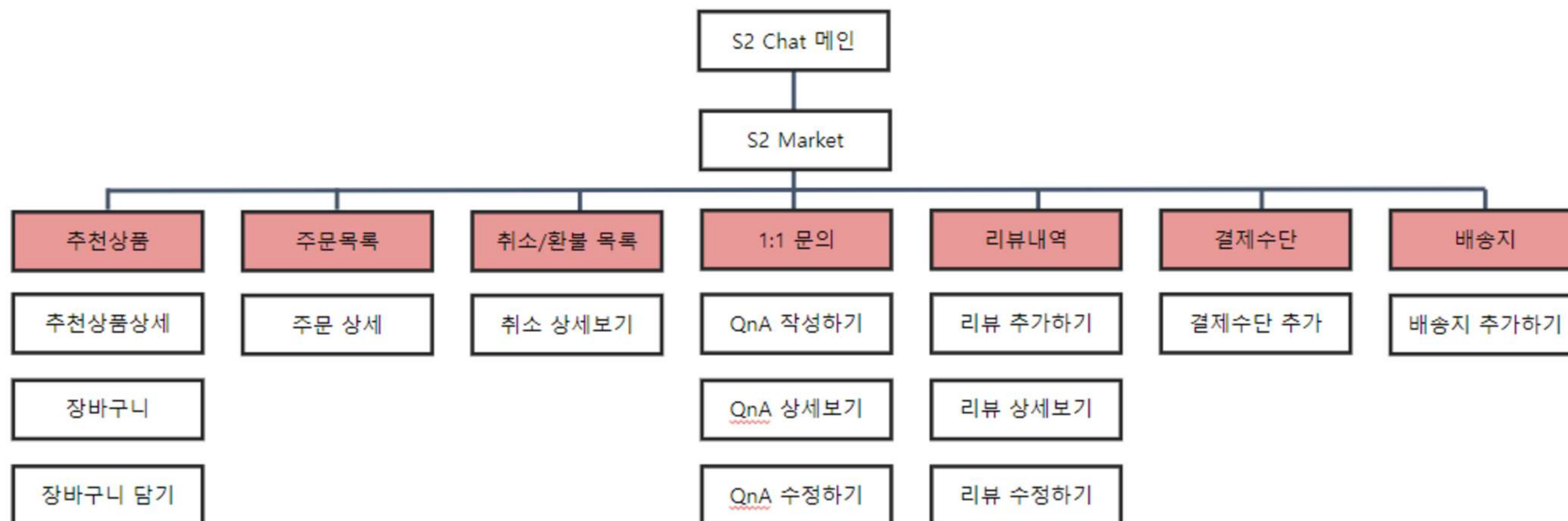
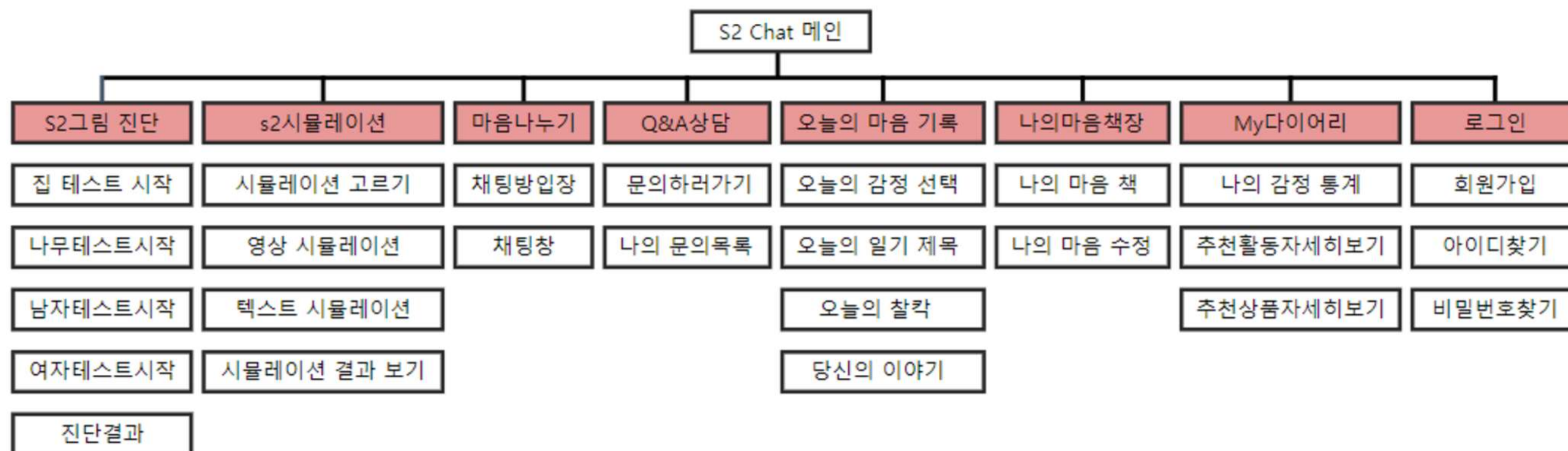
최종프로젝트 | ER 다이어그램

담당 : 윤수영



최종프로젝트 | 메뉴구성도

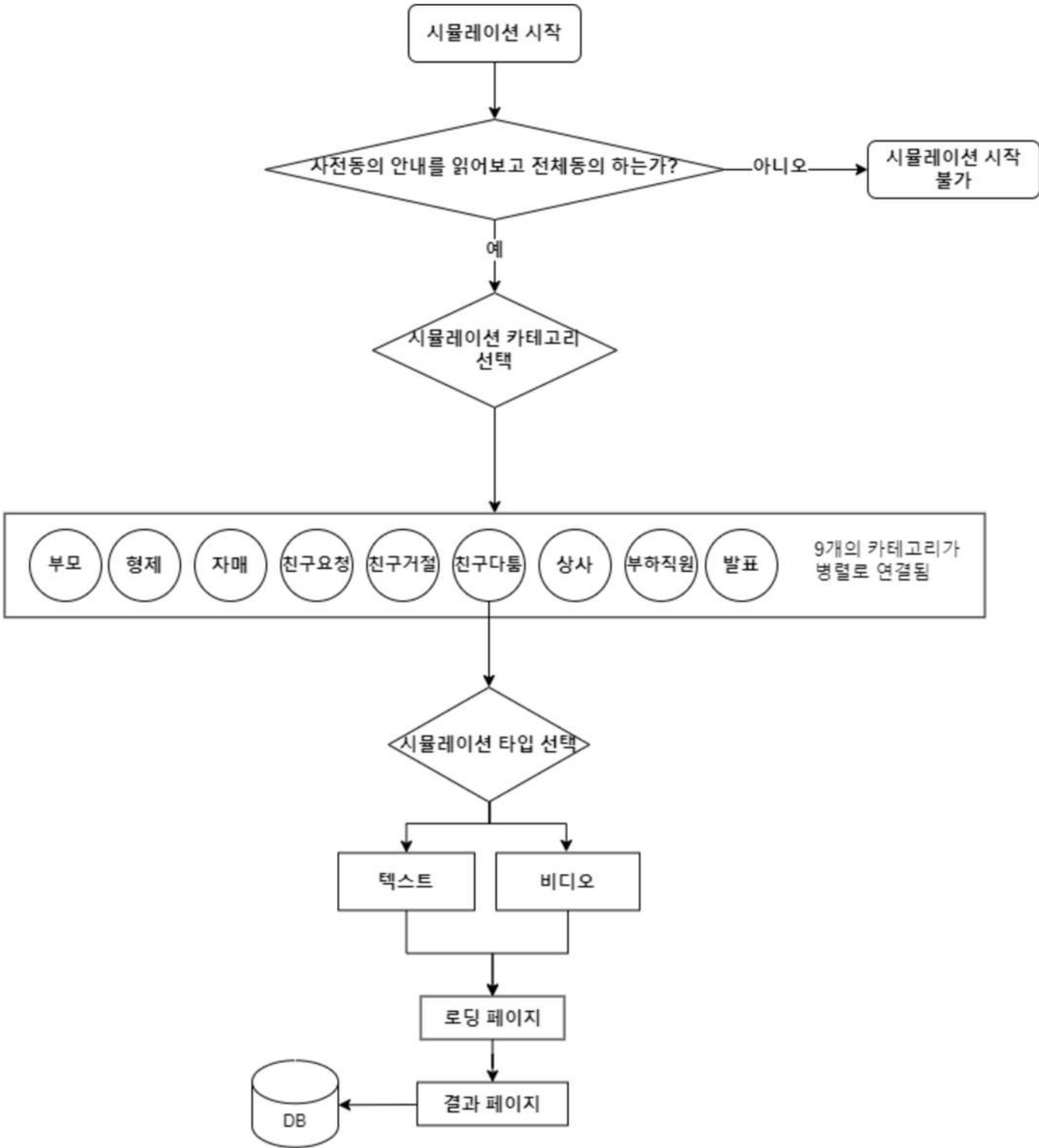
담당 : 윤수영



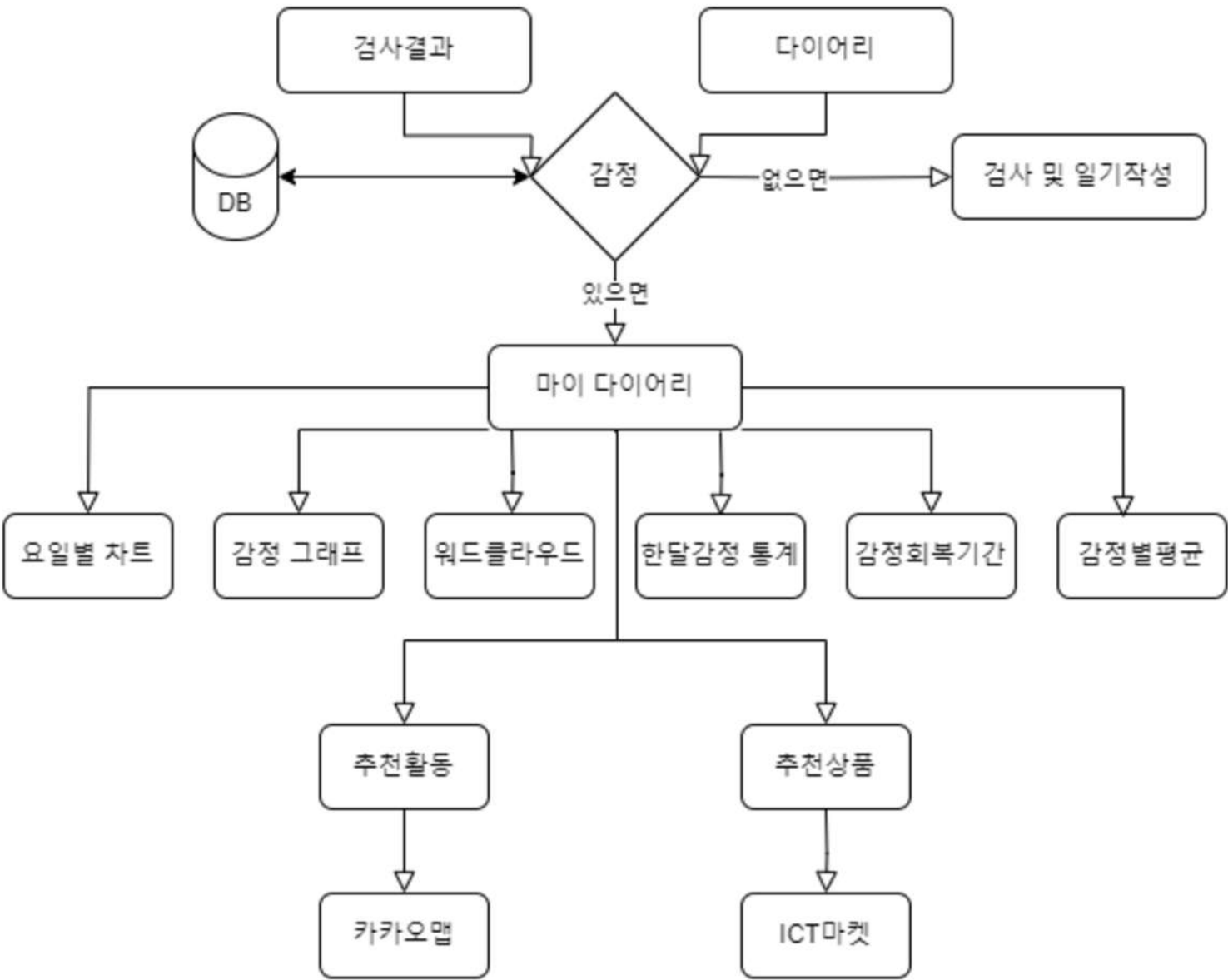
최종프로젝트 | 플로우차트

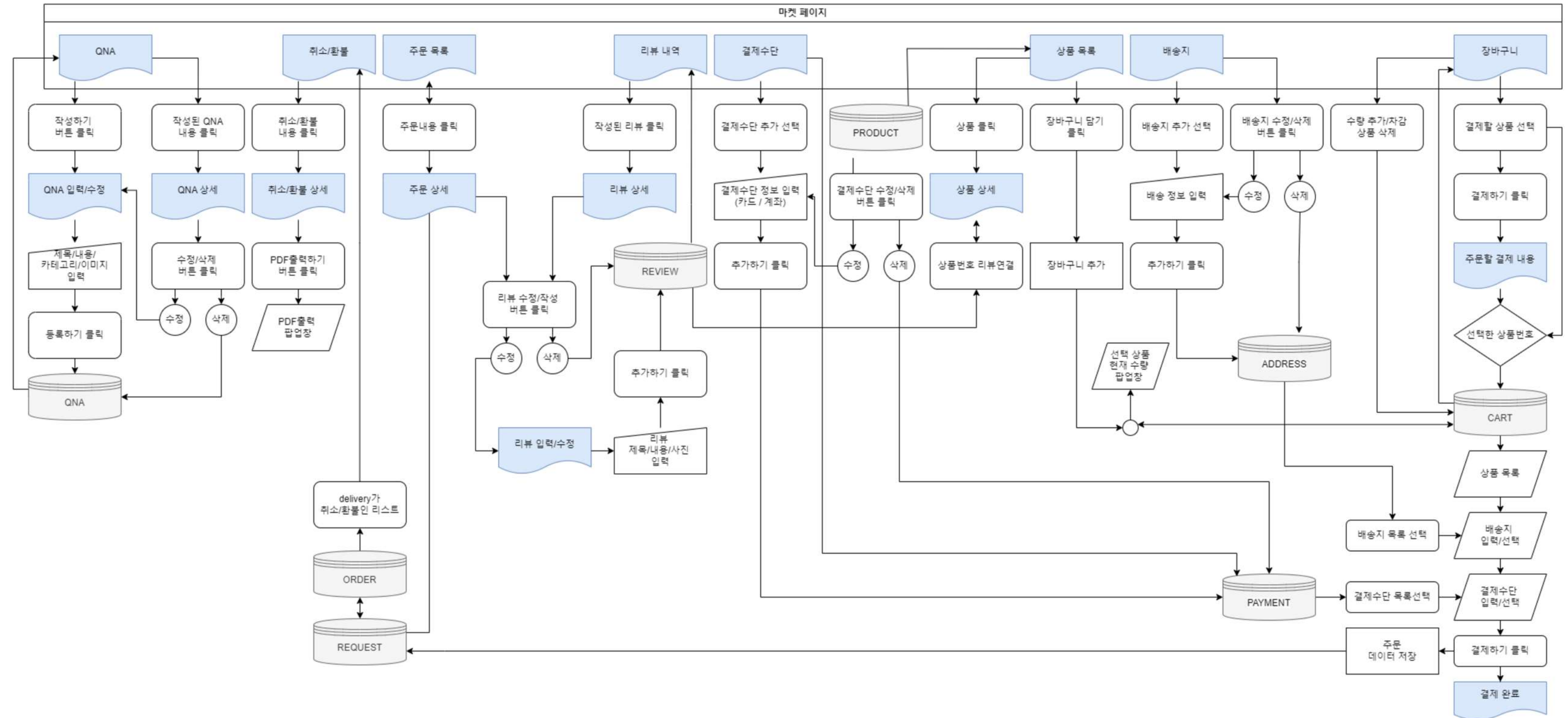
담당 : 윤수영

시뮬레이션



마이다이러리 (감정통계 부분)





최종프로젝트 | 실행화면

마이 다이어리 - 작성 화면

감정일기장

오늘의 감정 선택

행복하다

신난다

설렌다

고맙다

반갑다

부끄럽다

만족스럽다

최남다

속상하다

짜증난다

불안하다

상처받았다

감동했다

슬프다

피곤하다

지친다

후회된다

억울하다

외롭다

긴장된다

미안하다

놀랐다

걱정된다

상심하다

귀찮다

다음

감정일기장

오늘의 일기 제목

날씨가 우중충해요

이전

다음

감정일기장

오늘의 찰칵

파일 선택 turtle.png



이전

다음

감정일기장

당신의 이야기를 들려주세요

오늘 갑자기 면접이 잡혔어요. 면접을 보러가야하는데 정장이 없어요. 그래서 갑자기 정장이 필요해 취업날개 서비스를 신청했어요. 그래서 먼저 혼자 집에 가야해요. 그렇다구요.

이전

저장

오늘의 감정 선택 일기 제목 오늘의 사진 일기 쓰기

마이 다이어리 - 조회, 수정 화면

나의 마음 책

나의 일기장

Page: 1 of 5

나의 마음 책

날씨가 우중충해요

오늘 갑자기 면접이 잡혔어요. 면접을 보러가야하는데 정장이 없어요. 그래서 갑자기 정장이 필요해 취업날개 서비스를 신청했어요. 그래서 먼저 혼자 집에 가야해요. 그렇다구요. 귀찮다 긴장된다 피곤하다



2024년 8월 26일 월요일

수정

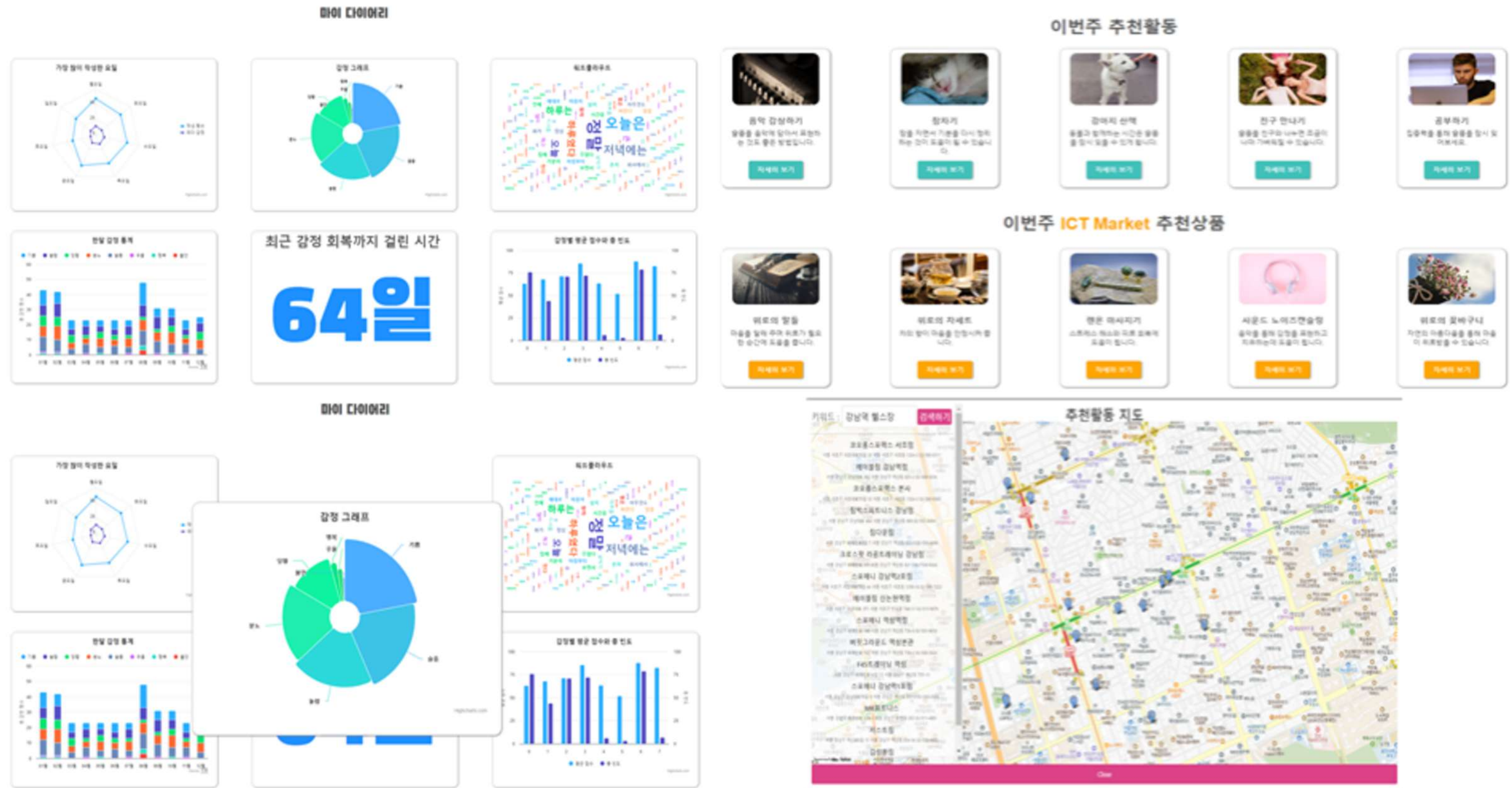
삭제

Page: 4 of 5

일기 표시 일기 읽기, 수정, 삭제

최종프로젝트 | 실행화면

마이페이지 - 나의 감정 통계, 추천 서비스



최종프로젝트 | 실행화면

S2 Market - 쇼핑몰, 장바구니 화면

실상 마켓

추천상품

주문목록

취소/환불목록

1:1문의

리뷰내역

결제수단

배송지

상품목록

전체8

진행4

불안2

유료2



【디퓨저】 나른한 주말 오후 햇살같은 따뜻한 향의 디퓨저

₩18,900

장바구니 담기



【티】 나른한 오후를 깨워줄 국내산 티백 녹차

₩3,000

장바구니 담기



【커피】 스테비아로 달 걱정을 줄인 커피믹스

₩12,000

장바구니 담기



【비타민】 우울한 기분을 달래줄 햇살같은 비타민 D 30일분

₩10,000

장바구니 담기



【관】 건강한 체력을 위한 침향환 골드 선물 세트

₩69,000

장바구니 담기



【호소】 장 건강과 피부 건강을 한번에 챙겨주는 곡물 호소 프로바이오틱스

₩23,100

장바구니 담기



【티】 심신 안정에 도움을 주는 국내산 티백 세트

₩32,000

장바구니 담기



【디퓨저】 달달한 향으로 기분을 편안하게 하는 디퓨저

₩19,800

장바구니 담기

실상 마켓

추천상품

주문목록

취소/환불목록

1:1문의

리뷰내역

결제수단

배송지

장바구니

상품목록

	상품이미지	상품명	수량	가격
☒		【티】 심신 안정에 도움을 주는 국내산 티백 세트	- 1 + 삭제	₩32,000
☐		【환】 건강한 체력을 위한 침향환 골드 선물세트	- 1 + 삭제	₩69,000
☒		【비타민】 우울한 기분을 달래줄 햇살같은 비타민 D 30일분	- 1 + 삭제	₩10,000

결제하기

최종프로젝트 | 구현 코드

RESTful API 설계

- RESTful API 원칙 적용
 - HTTP 메서드(GET, POST, PUT, DELETE) 활용
- URI 설계
 - 클라이언트-서버 구조를 유지하며 URI 설계를 직관적으로 구성

```
@RestController
@RequestMapping("/api/test")
public class SimpletestController {

    @Autowired
    private SimpletestService simpletestService;

    @GetMapping("/questions")
    public ResponseEntity> getQuestions() {
        return ResponseEntity.ok(simpletestService.getQuestion());
    }
}
```

최종프로젝트 | 구현 코드

DAO + Service 패턴을 활용한 데이터 관리

- DAO (Data Access Object) 패턴 적용
 - DAO 패턴을 적용하여 데이터베이스와의 직접적인 상호작용을 분리하고, 데이터 액세스를 캡슐화
 - SimpletestDAO 클래스는 MyBatis와 연동하여 SQL 쿼리를 실행하고 결과를 반환함
- Service 계층의 역할
 - Service 계층은 DAO에서 데이터를 받아와 비즈니스 로직을 수행하는 역할을 담당
 - Controller → Service → DAO → Database 구조로 데이터 흐름을 제어함

```
@Service
public class SimpletestService {
    @Autowired
    private SimpletestDAO simpletestDAO;

    public List<SimpletestVO> getQuestion() {
        return simpletestDAO.getQuestion();
    }
}
```

최종프로젝트 | 구현 코드

CORS 설정을 통한 보안 강화

- 프론트엔드와 백엔드의 도메인이 다를 경우 발생하는 CORS 문제를 해결
- 특정 도메인에서만 요청을 허용하여 불법적인 API 요청을 차단

```
@Bean
public WebMvcConfigurer corsConfigurer() {
    return new WebMvcConfigurer() {
        @Override
        public void addCorsMappings(CorsRegistry registry) {
            registry.addMapping("/**")
                .allowedOrigins("http://localhost:3000", "http://192.168.0.39:3000")
                .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS");
        }
    };
}
```


최종프로젝트 | 구현 코드

예외 처리 및 로깅 적용

1. 예외처리

- 서비스에서 발생하는 예외를 try-catch 블록으로 처리하여 애플리케이션의 안정성을 확보

2. 로깅 시스템

- SLF4J 기반 로깅 시스템을 활용하여 발생한 오류를 기록

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

private static final Logger logger =
    LoggerFactory.getLogger(SimpletestService.class);

public void insertResponse(UserResponseVO userResponseVO) {
    try {
        simpletestDAO.insertResponse(userResponseVO);
    } catch (Exception e) {
        logger.error("데이터 삽입 중 오류 발생: {}", e.getMessage());
        throw new RuntimeException("Error inserting response", e);
    }
}
```

최종프로젝트 | 구현 코드

DAO 패턴 (Data Access Object)

- SimpletestDAO를 활용하여 데이터베이스와의 상호작용을 캡슐화하여 유지보수성을 향상
- @Mapper 어노테이션을 사용하여 MyBatis와 연동된 데이터베이스 접근 객체로 설정
- 데이터베이스와 직접 상호작용하는 메서드 제공
 - findAll(), insertSimulation(), updateSimulation(), deleteSimulation()
- 특정 시뮬레이션의 대화 및 감정 데이터를 조회하는 findSimulationWithDialogAndEmotion() 메서드 포함

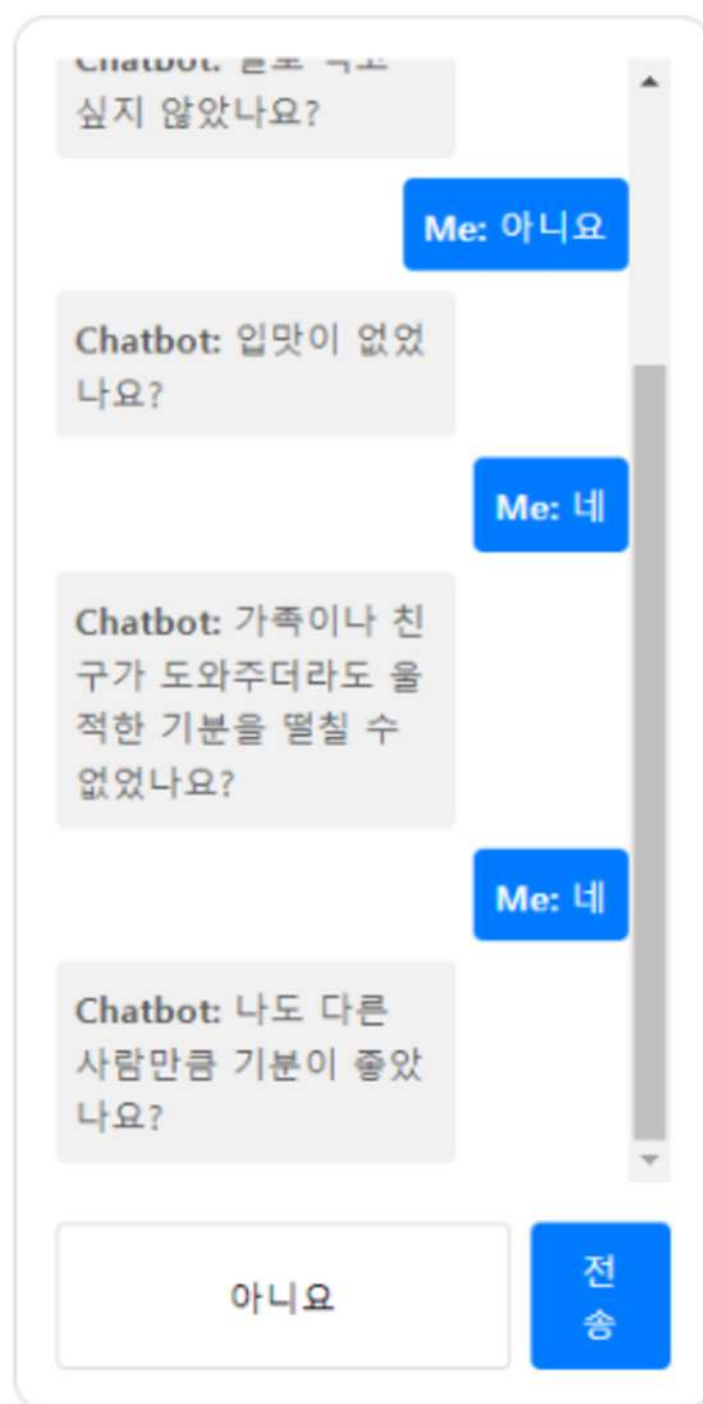
```
// DAO 패턴 적용: 데이터베이스와 직접 상호작용하는 인터페이스
@Mapper
public interface SimpletestDAO {
    // 설문 질문 목록 조회 (SELECT)
    List<SimpletestV0> getQuestion();
    // 사용자의 설문 응답 저장 (INSERT)
    void insertResponse(UserResponseV0 userResponseV0);
    // 특정 사용자의 특정 날짜 설문 응답 상세 조회 (SELECT)
    List<SimpletestV0> getRespDetail(@Param("userid") String userid, @Param("respdate")
String respdate);
    // 특정 사용자의 특정 날짜 설문 점수 총합 조회 (SELECT)
    int getRespTotal(@Param("userid") String userid, @Param("respdate") String respdate);
    // 특정 사용자의 특정 날짜 설문 응답 목록 조회 (SELECT)
    List<UserResponseV0> getUserResponses(@Param("userid") String userid,
@Param("respdate") String respdate);
    // 특정 사용자의 모든 설문 응답 목록 조회 (SELECT)
    List<Map<String, Object>> getUserResponses(@Param("userid") String userid)
}
```


최종프로젝트 | 구현 코드

담당 : 윤수영

CES-D 설문을 활용한 심리진단 서비스

- 채팅형 심리진단 서비스 (Controller, Vue (TTS))



```

@RestController
@RequestMapping("/survey")
public class SimpletestController {
    @GetMapping("/getRespTotal")
    public int getRespTotal(@RequestParam("userid") String userid,
                           @RequestParam("respdate") String respdate) {
        int score = simpletestService.getRespTotal(userid, respdate);
        int state = 0;
        if (10 <= score) {
            state = 1;
        } else if (5 <= score) {
            state = 2;
        } else if (0 <= score) {
            state = 3;
        } else {
            state = 0;
        }
        return state;
    }
}

@GetMapping("/responses")
public List<Map<String, Object>> getUserResponses(@RequestParam("userid") String userId) {
    return simpletestService.getUserResponses(userId);
}

async fetchSurveyQuestions() {
    try {
        const response = await
        axios.get(`${process.env.VUE_APP_BACKEND_URL}/survey/getQuestion`);
        this.surveyQuestions = response.data;
        this.addChatbotMessage(this.surveyQuestions[this.currentQuestionIndex].question);
    }

    // 텍스트를 음성으로 변환해주는 함수
    speak(text) {
        const msg = new SpeechSynthesisUtterance(text);
        msg.lang = 'ko-KR'; // 한국어 설정
        // 음성을 재생
        window.speechSynthesis.speak(msg);
    },

```


최종프로젝트 | 구현 코드

담당 : 윤수영

CES-D 설문을 활용한 심리진단 서비스

- 사용자의 응답을 정수형 점수로 환산 (Vue)



```
// 사용자 응답을 서버에 저장하는 함수
async sendResponses() {
  try {
    const userid = 'user2';
    const responseDate = new Date().toISOString().split('T')[0];
    for (const response of this.responses) {
      await axios.post(`${process.env.VUE_APP_BACKEND_URL}/survey/insertResponse`, {
        userid: userid,
        surveycode: this.surveyCode,
        qnum: response.qnum,
        respscore: response.respscore,
        respdate: responseDate
      });
    }
  } catch (error) {
    console.error('Error saving responses:', error);
  }
}
```

```
// 사용자가 입력한 메시지의 점수를 계산하는 함수
calculateScore(text) {
  const positiveWords = ['고맙', '내', '용', '매우', '엄청', '맞아'];
  const lowerText = text.toLowerCase();
  return positiveWords.some(word => lowerText.includes(word)) ? 1 : 0;
},
// 메시지 창을 자동으로 스크롤하는 함수
scrollToEnd() {
  const container = this.$el.querySelector('.messages');
  container.scrollTop = container.scrollHeight;
},
startRecording() {
  navigator.mediaDevices.getUserMedia({ audio: true })
    .then(stream => {
      this.mediaRecorder = new MediaRecorder(stream);
      this.mediaRecorder.ondataavailable = event => {
        this.audioChunks.push(event.data);
      };
      this.mediaRecorder.onstop = this.handleStopRecording;
      this.mediaRecorder.start();
      this.isRecording = true;
    })
    .catch(error => {
      console.error('Error accessing media devices:', error);
      // 사용자에게 오류 메시지 표시 또는 다른 대응 방안 구현
    });
},
```

최종프로젝트 | 구현 코드

담당 : 윤수영

CNN을 활용한 안면이미지 표정 감정 분석 모델

• 데이터 확인



#감정 분석

#표정 분석

한국인 감정인식을 위한 복합 영상

분야

영상이미지

유형

이미지

구축년도 : 2020 갱신년월 : 2023-10 조회수 : 32,078 다운로드 : 4,723 용량 : 835.61 GB

다운로드

↓ 샘플 데이터 ?

관심데이터 등록 155

소개

파일 목록 (API 다운로드)

카테고리(종)	이미지 개수(개)	분포(%)
기쁨	70,735	14.47%
당황	70,457	14.41%
분노	68,835	14.08%
불안	69,965	14.31%
상처	70,103	14.34%
슬픔	70,508	14.42%
중립	68,173	13.94%
평균	69,825	14.28

```
expressions = os.listdir(folder_path)

expression_counts = []
for expression in expressions:
    expression_path = os.path.join(folder_path, expression)
    num_images = len(os.listdir(expression_path))
    print(f"{expression} : {num_images} images")
    expression_counts.append(num_images)

anger : 8000 images
anxiety : 8000 images
happy : 8000 images
naturally : 8592 images
panic : 8000 images
sad : 8000 images
wound : 8016 images

folder_path2 = '/content/drive/MyDrive/Colab Notebooks/ICTStudy/f
expressions = os.listdir(folder_path2)
expression_counts = []
for expression in expressions:
    expression_path = os.path.join(folder_path2, expression)
    num_images = len(os.listdir(expression_path))
    print(f"{expression} : {num_images} images")
    expression_counts.append(num_images)

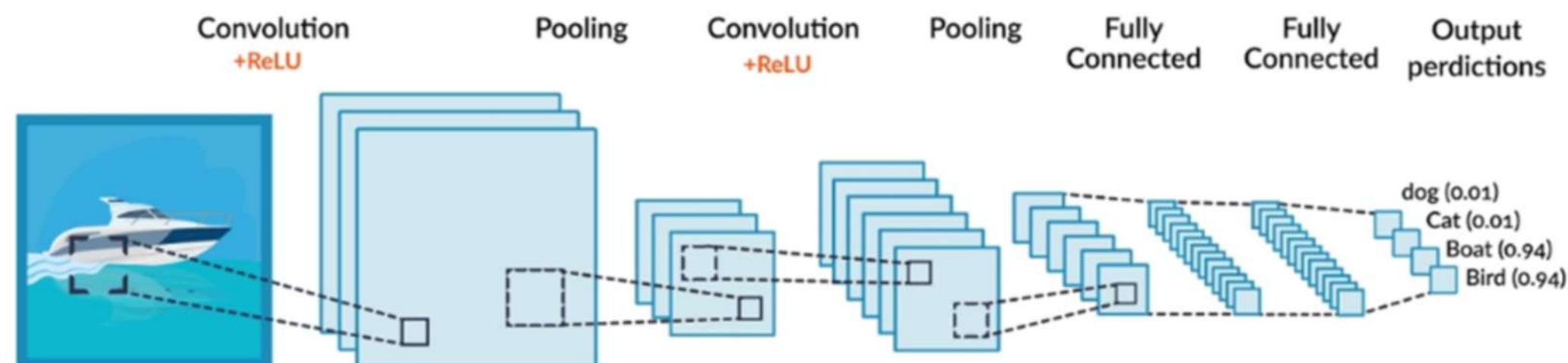
anger : 2000 images
anxiety : 2000 images
happy : 2000 images
naturally : 2595 images
panic : 2000 images
sad : 2000 images
wound : 2022 images
```


최종프로젝트 | 구현 코드

담당 : 윤수영

CNN을 활용한 안면이미지 표정 감정 분석 모델

• CNN모델 사용 이유



컨볼루션 신경망
(Convolutional Neural Network)

- 합성곱 신경망(CNN), 합성곱 연산 사용
- 3차원 데이터의 공간적 정보 유지
- 동물의 시신경 구조와 유사하게 뉴런 사이의 연결 패턴 형성 모형
- 특징 지도(Feature Map)를 이용한 학습
- 컴퓨터 비전, 영상 분석 및 인식에 많이 사용

- 높은 정확도로 분류 작업 수행 가능
- 이미지의 로컬 패턴 추출하고 중요한 시각적 정보 강조
- 이미지 내의 중요한 구조와 패턴을 인식, 모델이 이미지나 비디오를 보다 효과적으로 분석할 수 있도록 도움

최종프로젝트 | 구현 코드

담당 : 윤수영

CNN을 활용한 안면이미지 표정 감정 분석 모델

- Training Data 70,000건을 사용하여 모델 구현

```
no_of_classes = len(emotions)
picture_size = 128
batch_size = 128
epochs = 100
l2_reg = 0.0001 # L2 정규화 강도 설정

# 모델 생성 함수
def create_model(input_shape=(picture_size, picture_size, 1), no_of_classes=no_of_classes,
                 l2_reg=l2_reg):
    model = Sequential()

    # 1층 - Conv2D -> BatchNorm -> Activation -> MaxPooling -> Dropout
    model.add(Conv2D(32, (3, 3), padding='same', activation='relu', input_shape=input_shape,
                    kernel_regularizer=l2(l2_reg)))
    model.add(BatchNormalization())
    model.add(MaxPooling2D((2, 2)))
    model.add(Dropout(0.3))

    # 2층 - Conv2D -> BatchNorm -> Activation -> MaxPooling -> Dropout
    model.add(Conv2D(64, (3, 3), padding='same', activation='relu',
                    kernel_regularizer=l2(l2_reg)))
    model.add(BatchNormalization())
    model.add(MaxPooling2D((2, 2)))
    model.add(Dropout(0.3))

    # 3층 - Conv2D -> BatchNorm -> Activation -> MaxPooling -> Dropout
    model.add(Conv2D(128, (3, 3), padding='same', activation='relu',
                    kernel_regularizer=l2(l2_reg)))
    model.add(BatchNormalization())
    model.add(MaxPooling2D((2, 2)))
    model.add(Dropout(0.4))

    # 4층 - Conv2D -> BatchNorm -> Activation -> MaxPooling -> Dropout
    model.add(Conv2D(256, (3, 3), padding='same', activation='relu',
                    kernel_regularizer=l2(l2_reg)))
    model.add(BatchNormalization())
    model.add(MaxPooling2D((2, 2)))
    model.add(Dropout(0.4))
```

```
# CNN -> Flatten() -> DNN
model.add(Flatten())

# 5층 - Dense -> BatchNorm -> Activation -> Dropout
model.add(Dense(512, kernel_regularizer=l2(l2_reg)))
model.add(BatchNormalization())
model.add(Activation('relu'))
model.add(Dropout(0.5))

# 6층 - Dense -> BatchNorm -> Activation -> Dropout
model.add(Dense(512, kernel_regularizer=l2(l2_reg)))
model.add(BatchNormalization())
model.add(Activation('relu'))
model.add(Dropout(0.5))

# 출력층 - Dense -> Activation
model.add(Dense(no_of_classes, activation='softmax'))

# 모델 컴파일
model.compile(optimizer=Adam(learning_rate=0.0001),
              loss='categorical_crossentropy',
              metrics=['accuracy'])

return model
```

추가된 Dense층

- L2 정규화: 각 Conv2D와 Dense 층에서 `kernel_regularizer=l2(l2_reg)`를 사용하여 과적합을 방지
- 깊이와 복잡성 증가: 더 많은 필터(32, 64, 128, 256)와 더 큰 Dense 층(512)을 사용하여 네트워크의 표현력을 강화

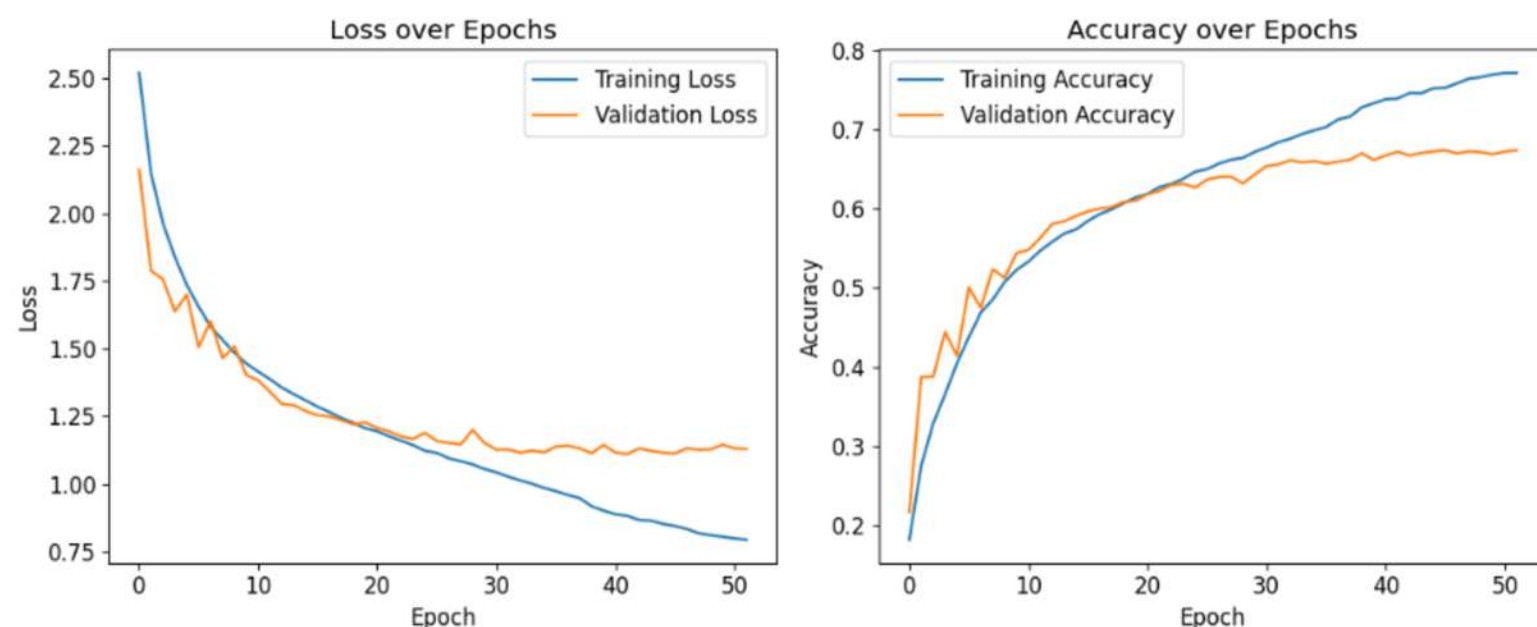
최종프로젝트 | 구현 코드

담당 : 윤수영

CNN을 활용한 안면이미지 표정 감정 분석 모델

- Training Data 70,000건을 사용하여 모델 구현

Test Accuracy: **67.319%**



```
import matplotlib.pyplot as plt

# 학습 및 검증 손실 시각화
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.title('Loss over Epochs')

# 학습 및 검증 정확도 시각화
plt.subplot(1, 2, 2)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.title('Accuracy over Epochs')

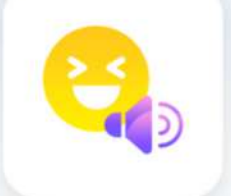
plt.tight_layout()
plt.show()
```

Epochs: 100
Batch Size: 128
Learning Rate: 0.0001
Test Accuracy: 67.31%

- 초기 테스트 정확도 62.31%에서 최종 테스트 정확도 67.32%로, 정확도는 약 5% 포인트 향상되었음
- 실패한 모델은 기본적인 CNN 구조로, 학습과정에서 모델 성능 제한적
- 성공한 모델은 더 깊고 복잡한 구조, L2 정규화 및 흑백 이미지 처리 등 다양한 최적화로 성능을 크게 개선했음

KoBert 를 활용한 문장 내 감정 인식

• 데이터 확인



#한국어 대화 음성 #감정 라벨링 #다분류 감정

감정 분류를 위한 대화 음성 데이터셋

분야 미분류 유형 오디오

조회수: 16,222 다운로드: 1,486 용량: 14.41 GB

데이터 통계

1차 공개 구축량

- 4차년도 14,606건
- 5차년도 10,011건
- 5차년도(2차) 19,374건

1.데이터 로드 및 컬럼명 확인

```
[14] import pandas as pd

path = '/content/merge_data.csv'
data = pd.read_csv(path, encoding='cp949')
```

```
[15] data.columns

Index(['wav_id', '발화문', '상황', '1번 감정', '1번 감정세기', '2번 감정', '2번 감정세기', '3번 감정', '3번 감정세기', '4번 감정', '4번감정세기', '5번 감정', '5번 감정세기', '나이', '성별'],
      dtype='object')
```

2.감정 라벨 데이터 확인

```
[16] data_df = data[['발화문', '1번 감정', '2번 감정', '3번 감정', '4번 감정', '5번 감정']]
data_df.head(3)
```

	발화문	1번 감정	2번 감정	3번 감정	4번 감정	5번
0	개를 예쁘다고 사놓고 끝까지 키우지도 않고 버리는 사람들이 엄청 많아졌대.	Angry	Angry	Angry	Angry	/
1	지금도 그대로 있어. 치우는 사람이 없어.	Neutral	Disgust	Sadness	Disgust	Di
2	맞아. 무기력증인 것 같아. 한동안 정말 바빴었거든.	Sadness	Sadness	Sadness	Sadness	Sa

3.감정 라벨 개수 확인

```
data_df['1번 감정'].value_counts()
```

	count
1번 감정	
Sadness	3998
Neutral	2498
Disgust	1212
Fear	1114
Angry	1012
Happiness	121
Surprise	56

dtype: int64

4.감정 라벨 벡터화

```
# 감정 클래스 레이블을 숫자로 변환
emotion_mapping = {
    "Fear": 0,
    "Surprise": 1,
    "anger": 2,
    "Angry": 2,
    "sad": 3,
    "Sadness":3,
    "Neutral": 4,
    "Happiness": 5,
    "Disgust": 6
}
```

```
data_df['1번 감정'] = data_df['1번 감정'].map(emotion_mapping)
data_df['1번 감정'].value_counts()
```

<ipython-input-22-ba4a5386ae29>:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: <https://pandas.pydata.org/pandas-docs/stable/10min.html#copy-on-write>
data_df['1번 감정'] = data_df['1번 감정'].map(emotion_mapping)

	count
1번 감정	
3	3998
4	2498
6	1212
0	1114
2	1012
5	121
1	56

dtype: int64

최종프로젝트 | 구현 코드

담당 : 윤수영

KoBert 를 활용한 문장 내 감정 인식

• 데이터 라벨 전처리

◦ 대, 소문자가 혼용되어 있어 모든 라벨을 소문자로 전처리

```
df1 = pd.read_csv('/content/5차년도.csv', encoding='cp949')
df2 = pd.read_csv('/content/4차년도.csv', encoding='cp949')
df3 = pd.read_csv('/content/5차년도_2차.csv', encoding='cp949')
```

```
print(df1['1번 감정'].value_counts())
print(df2['1번 감정'].value_counts())
print(df3['1번 감정'].value_counts())
```

```
1번 감정
Sadness      3998
Neutral      2498
Disgust       1212
Fear          1114
Angry         1012
Happiness      121
Surprise        56
Name: count, dtype: int64
```

```
1번 감정
Sadness      6593
Angry        3374
Neutral      2635
Disgust       913
Fear          645
Happiness     322
Surprise      124
Name: count, dtype: int64
```

```
1번 감정
neutral      6825
happiness    3066
sadness      2824
disgust      2252
surprise     1739
fear         1351
angry        1317
Name: count, dtype: int64
```

데이터 결합

```
data_merge = pd.concat([df1, df2, df3])
```

'1번 감정' 컬럼의 값을 소문자로 변환

```
data_merge['1번 감정'] = data_merge['1번 감정'].str.lower()
```

변환된 결과를 확인하기 위해 value_counts()를 다시 실행

```
print(data_merge['1번 감정'].value_counts())
```

```
1번 감정
sadness      13415
neutral      11958
angry         5703
disgust       4377
happiness     3509
fear          3110
surprise      1919
Name: count, dtype: int64
```

```
data_merge['1번 감정'].value_counts()
```

```
count
1번 감정
sadness  13415
neutral  11958
angry    5703
disgust  4377
happiness 3509
fear     3110
surprise 1919
```

```
dtype: int64
```

최종프로젝트 | 구현 코드

담당 : 윤수영

KoBert 를 활용한 문장 내 감정 인식

• 데이터 증강 (KoBert를 사용하여 단어 위치를 조정하여 다양화)

```
if __name__ == "__main__":
    # 데이터 불러오기
    df = pd.read_csv(f'{path}/emotion_vec.csv')

    # 각 라벨의 타겟 개수 설정
    target_count = 14000
    iter_count = 2 # 각 텍스트에 대해 증강 작업을 몇 번 수행할지 설정

    # 각 라벨에 대해 증강 수행
    unique_labels = df['label'].unique()
    for label in unique_labels:
        df = textAug(df, label, target_count, iter_count)

    filename = f'{path}/textaug_completed_dataframe_0808.csv'
    df.to_csv(filename, index=False)
    print(f'Data augmentation completed. Saved to {filename}')
```

```
Augmentations: 0%|          | 0/2 [00:00<?, ?it/s]
Augmentations: 50%|██████    | 1/2 [00:00<00:00, 6.35it/s]
Augmentations: 100%|██████████| 2/2 [00:00<00:00, 6.70it/s]
Rows Processing: 46%|██████    | 2087/4548 [1:34:38<29:53, 1.37it/s]
Augmentations: 0%|          | 0/2 [00:00<?, ?it/s]
Augmentations: 50%|██████    | 1/2 [00:02<00:02, 2.89s/it]
Augmentations: 100%|██████████| 2/2 [00:04<00:00, 2.20s/it]
Rows Processing: 46%|██████    | 2088/4548 [1:34:43<1:17:42, 1.90s/it]
```

```
df = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/ICTStudy/ATeam/incredata/emotionData_final.csv')
df['label'].value_counts()
```

	count
label	
0	16000
3	16000
2	16000
5	16000
6	16000
4	16000
1	16000

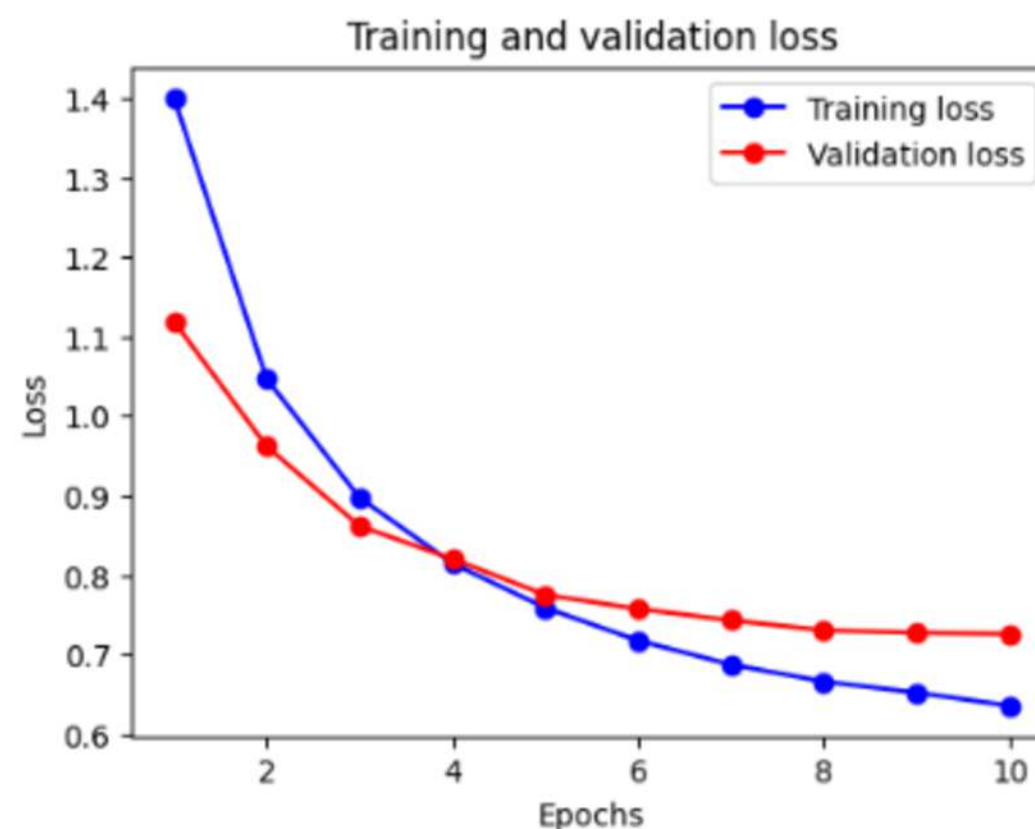
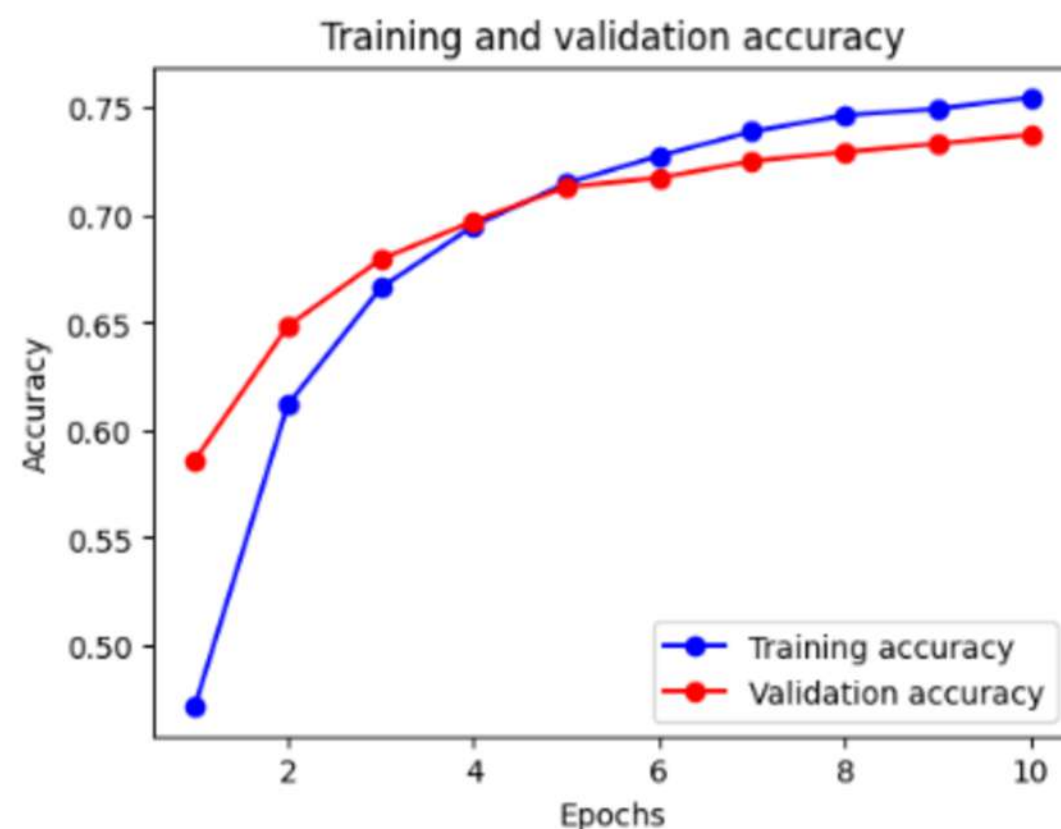
- 라벨 값 비율이 고르지 못한 것을 확인 후 텍스트 데이터 증강
- 3000 ~ 4000 개 사이 값을 가지는 라벨이 많아 4000개 기준으로 증강 후 10000개로 다시 증강 시도함
- 런타임 유지를 위해 100개씩 40번 반복, 100개씩 100번 반복함

최종프로젝트 | 구현 코드

담당 : 윤수영

KoBert 를 활용한 문장 내 감정 인식

- 데이터 증강한 데이터 셋으로 재학습 (과대적합으로 확인되어 잘못된 학습 확인)



```
learning(max_len=64,  
batch_size=32,  
warmup_ratio=0.03,  
num_epochs=10,  
max_grad_norm=1,  
log_interval=200,  
learning_rate=3e-5)
```

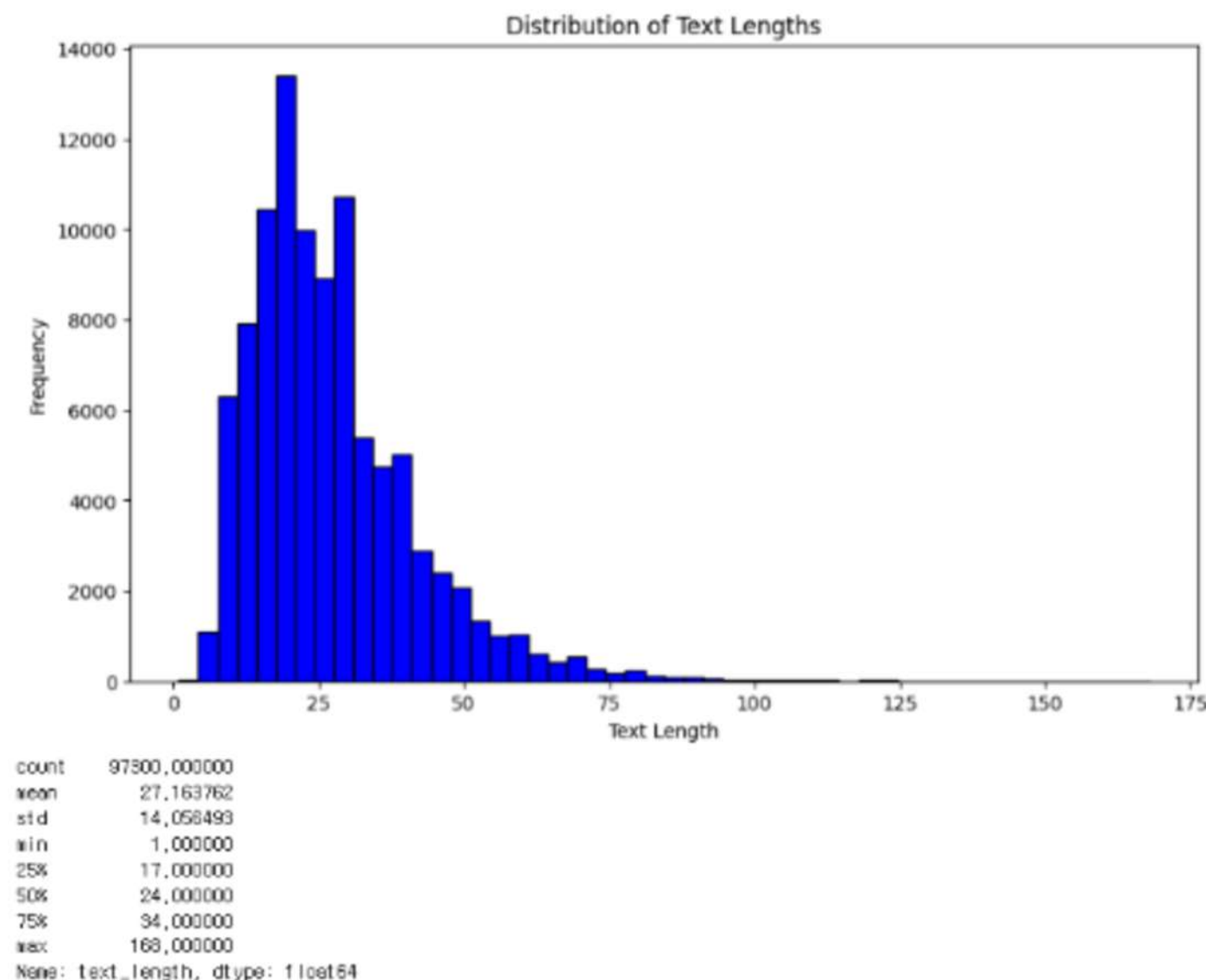
- 동일한 파라미터로 학습
- 여러번 테스트 하기 위해 학습, 평가, 시각화 코드를 함수화함

최종프로젝트 | 구현 코드

담당 : 윤수영

KoBert 를 활용한 문장 내 감정 인식

• 데이터 정보 확인 (최대 길이 수치 확인)



```
# 데이터 로드
data = pd.read_csv(f'{path}/emotionData_final.csv')

# 텍스트 길이 계산
data['text_length'] = data['text'].apply(len)

# 텍스트 길이 분포 시각화
plt.figure(figsize=(10, 6))
plt.hist(data['text_length'], bins=50, color='blue', edgecolor='black')
plt.title('Distribution of Text Lengths')
plt.xlabel('Text Length')
plt.ylabel('Frequency')
plt.show()

# 텍스트 길이의 통계값 확인
length_stats = data['text_length'].describe()
print(length_stats)
```

- 최소, 최대 텍스트 길이를 확인하고 조정해야 할 지 판단
- 25% ~ 75% 사이 길이를 가진 데이터를 추출하여 사용

최종프로젝트 | 구현 코드

담당 : 윤수영

KoBert 를 활용한 문장 내 감정 인식

- 데이터 전처리 (최소, 최대 텍스트에 대해 임의 값을 추가하거나 자름)

text

0 일단은 취소를 한번 해보고, 그게 안되면 약속 장소를 명동이 아닌 좀 가깝나 이런...

1 나 엘리베이터에 갇혔어.

2 식당에서 친구랑 밥먹고 있었거든. 근데 막 지진이 일어났어.

3 어디가 좋을까?

4 아, 너무 너무 무서워. 나 어쩌면 좋지? 119에 전화해봐야겠어.

cleaned_text

adjusted_text

	cleaned_text	adjusted_text
0	일단은 취소를 한번 해보고 그게 안되면 약속 장소를 명동이 아닌 좀 가깝나 이런 곳...	일단은 취소를 한번 해보고 그게 안되면 약속 장소를 명동이 아닌 좀 가깝...
1	나 엘리베이터에 갇혔어	나 엘리베이터에 갇혔어 [짧음]
2	식당에서 친구랑 밥먹고 있었거든 근데 막 지진이 일어났어	식당에서 친구랑 밥먹고 있었거든 근데 막 지진이 일어났어
3	어디가 좋을까	어디가 좋을까 [짧음]
4	아 너무 너무 무서워 나 어쩌면 좋지 에 전화해봐야겠어	아 너무 너무 무서워 나 어쩌면 좋지 에 전화해봐야겠어

```
learning(max_len=64,
        batch_size=32,
        warmup_ratio=0.03,
        num_epochs=10,
        max_grad_norm=1,
        log_interval=200,
        learning_rate=3e-5)

# 데이터 로드
data = pd.read_csv(f'{path}/emotionData_final.csv')

# 텍스트 정제 함수
def clean_text(text):
    text = re.sub(r'^가-있\s', '', text) # 특수 문자 제거
    text = re.sub(r'\s+', ' ', text) # 공백 정리
    return text.strip()

# 텍스트 길이 조정 함수
# def adjust_text_length(text, min_length=17, max_length=34): 25% ~ 75% 로 범위 조절
def adjust_text_length(text, min_length=17, max_length=34):
    length = len(text)

    if length < min_length:
        # 짧은 텍스트에 대해 특수 토큰을 추가하거나 보완
        return text + ' [짧음]'
    elif length > max_length:
        # 긴 텍스트는 자르거나 요약
        return text[:max_length] + '...'
    else:
        return text

# 데이터에 전처리 적용
data['cleaned_text'] = data['text'].apply(clean_text)
data['adjusted_text'] = data['cleaned_text'].apply(adjust_text_length)

# 원본 label과 함께 새로운 CSV 파일로 저장
output_file_path = 'adjusted_data_with_labels_0810.csv'
data[['text', 'cleaned_text', 'adjusted_text', 'label']].to_csv(output_file_path, index=False)

# 결과 확인
data[['text', 'cleaned_text', 'adjusted_text', 'label']].head()
```

최종프로젝트 | 구현 코드

담당 : 윤수영

KoBert 를 활용한 문장 내 감정 인식

- 데이터 전처리 (길이를 조정한 데이터의 성능 비교)

```
Original Data - Accuracy: 0.9304, F1 Score: 0.9304, Precision: 0.9307, Recall: 0.9304
Adjusted Data - Accuracy: 0.9254, F1 Score: 0.9255, Precision: 0.9262, Recall: 0.9254
```

```
# 모델 학습 및 예측 (원본 데이터)
model_orig = MultinomialNB()
model_orig.fit(X_train_orig_vec, y_train_orig)
y_pred_orig = model_orig.predict(X_test_orig_vec)

# 모델 학습 및 예측 (조정된 데이터)
model_adj = MultinomialNB()
model_adj.fit(X_train_adj_vec, y_train_adj)
y_pred_adj = model_adj.predict(X_test_adj_vec)

# 성능 평가 (원본 데이터)
accuracy_orig = accuracy_score(y_test_orig, y_pred_orig)
f1_orig = f1_score(y_test_orig, y_pred_orig, average='weighted')
precision_orig = precision_score(y_test_orig, y_pred_orig, average='weighted')
recall_orig = recall_score(y_test_orig, y_pred_orig, average='weighted')

# 성능 평가 (조정된 데이터)
accuracy_adj = accuracy_score(y_test_adj, y_pred_adj)
f1_adj = f1_score(y_test_adj, y_pred_adj, average='weighted')
precision_adj = precision_score(y_test_adj, y_pred_adj, average='weighted')
recall_adj = recall_score(y_test_adj, y_pred_adj, average='weighted')

# 결과 비교 출력
print("Original Data - Accuracy: {:.4f}, F1 Score: {:.4f}, Precision: {:.4f}, Recall: {:.4f}".format(accuracy_orig, f1_orig, precision_orig, recall_orig))
print("Adjusted Data - Accuracy: {:.4f}, F1 Score: {:.4f}, Precision: {:.4f}, Recall: {:.4f}".format(accuracy_adj, f1_adj, precision_adj, recall_adj))
```

- 데이터 재확인 : 텍스트 길이 조정이 모델 성능에 미치는 영향 비교
- 원본 수치가 더 좋아서 원본 데이터를 사용하기로 함

최종프로젝트 | 구현 코드

담당 : 윤수영

KoBert 를 활용한 문장 내 감정 인식

• Max Length 중앙 값으로 구성하여 학습 진행

```
Model saved at epoch 9 with test accuracy: 0.7300347222222222
Training Epoch 10: 0%|          | 1/609 [00:00<08:15, 1.23it/s]epoch 10 batch id 1 loss 0.7101476192474365 train acc 0.71875
Training Epoch 10: 33%|■■■■|    | 201/609 [01:48<03:39, 1.86it/s]epoch 10 batch id 201 loss 0.7512655854225159 train acc 0.7475124378109452
Training Epoch 10: 66%|■■■■■■|  | 401/609 [03:36<01:52, 1.84it/s]epoch 10 batch id 401 loss 0.7042497992515564 train acc 0.7448176433915212
Training Epoch 10: 99%|■■■■■■■■| 601/609 [05:24<00:04, 1.86it/s]epoch 10 batch id 601 loss 0.6305090188980103 train acc 0.7455282861896838
Training Epoch 10: 100%|■■■■■■■■| 609/609 [05:28<00:00, 1.86it/s]
epoch 10 train acc 0.7456896551724138 train loss 0.657299041062936
Evaluating Epoch 10: 100%|■■■■■■■■| 153/153 [00:27<00:00, 5.60it/s]
epoch 10 test acc 0.7340175653594772 test loss 0.7191685075853386
```

GPU 메모리 사용량: 1.46 GB

Model saved at epoch 10 with test accuracy: 0.7340175653594772

```
# 학습 파라미터 설정 예시
max_len = 128 # 또는 256, 512 등 실험을 통해 최적화
batch_size = 128 # GPU 메모리 상황에 따라 128, 256, 512
learning_rate = 2e-5 # 또는 3e-5, 5e-5 등 실험적으로 최적화
num_epochs = 10 # 충분한 학습을 위해 설정, 조기 종료에 있을 경우 적당한 값 설정
max_grad_norm = 1.0 # Gradient clipping을 위한 최대 값 설정
log_interval = 200 # 학습 로그를 출력할 간격 설정
warmup_ratio = 0.1 # 학습 초기 단계에서의 warmup 비율 설정
learning(max_len, batch_size, warmup_ratio, num_epochs, max_grad_norm, log_interval, learning_rate)
```

```
import torch

# 가상으로 모델과 데이터 로드
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = BERTClassifier(bertmodel, dr_rate=0.5).to(device)
dataset_train, dataset_test = train_test_split(data_list, test_size=0.2, shuffle=True,
random_state=32)
data_train = BERTDataset(dataset_train, 0, 1, tokenizer, max_len, True, False)

# 메모리 체크를 위해 임의의 배치 크기 설정
batch_size = 64 # 초기 설정, 이후 조정 가능

train_dataloader = DataLoader(data_train, batch_size=batch_size, num_workers=5,
shuffle=True)

# GPU 메모리 점검
print(f"GPU 메모리 사용량: {torch.cuda.memory_allocated() / 1024 ** 3:.2f} GB")
```

최종프로젝트 | 구현 코드

담당 : 윤수영

KoBert 를 활용한 문장 내 감정 인식

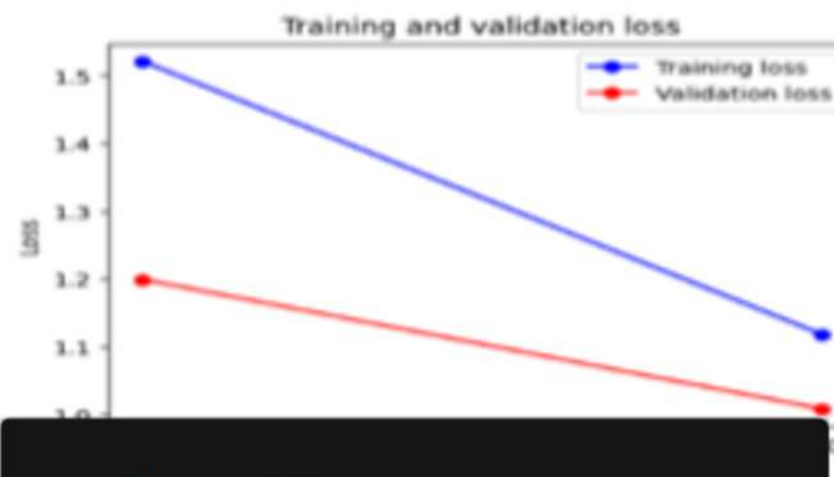
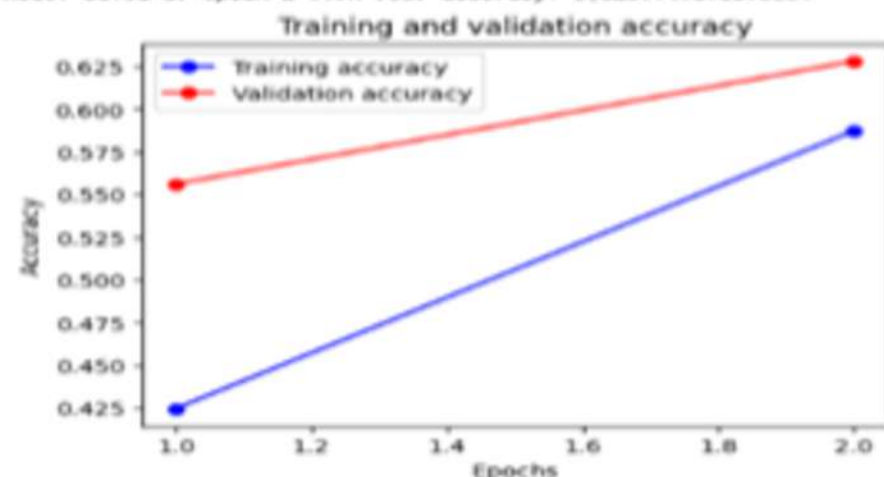
- Max Length 중앙 값으로 구성하여 학습 진행 (GPU 메모리 사용량 확인)

Training Epoch 2: 100%  609/609 [05:28<00:00, 1.85it/s]
 epoch 2 batch id 1 loss 1.2016433477401733 train acc 0.5390625
 epoch 2 batch id 201 loss 1.1662259101867676 train acc 0.5621113184079602
 epoch 2 batch id 401 loss 1.0336960554122925 train acc 0.575475374064838
 epoch 2 batch id 601 loss 0.9977717399597168 train acc 0.5865484608985025
 epoch 2 train acc 0.5873614532019704 train loss 1.118859624040538
 Evaluating Epoch 2: 100%  153/153 [00:27<00:00, 5.66it/s]
 epoch 2 test acc 0.6281147875816994 test loss 1.0085619765948626

GPU 메모리 사용량: 3.21 GB

조기 종료

Model saved at epoch 2 with test accuracy: 0.6281147875816994



```
max_len = 128
batch_size = 128
learning_rate = 3e-5
num_epochs = 10
max_grad_norm = 1.0
log_interval = 200
warmup_ratio = 0.1
```

```
model_name = 'test_kobert_early2'
max_len = 128
batch_size = 128
learning_rate = 3e-5
for i in range(len(list_sentence)):
    print(predict(list_sentence[i], model_name, max_len, batch_size))
```

/usr/local/lib/python3.10/dist-packages/huggingface_hub/utils/_token.py:89: UserWarning:
 The secret 'HF_TOKEN' does not exist in your Colab secrets.
 To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>).
 You will be able to reuse this secret in all of your notebooks.
 Please note that authentication is recommended but still optional to access public models or datasets:
 warnings.warn()

tokenizer_config.json: 100%  51.0/51.0 [00:00<00:00, 3.60kB/s]
 vocab.txt: 100%  77.8k/77.8k [00:00<00:00, 3.56MB/s]
 config.json: 100%  426/426 [00:00<00:00, 30.7kB/s]
 models.safetensors: 100%  369M/369M [00:01<00:00, 266MB/s]
 내용 : 우와 무지개다 대박 >> 입력하신 내용에서 놀람이 느껴집니다.
 내용 : 여기 조금 무섭다 >> 입력하신 내용에서 놀람이 느껴집니다.
 내용 : 진짜 예쁘다 >> 입력하신 내용에서 공포가 느껴집니다.
 내용 : 나 핸드폰 잃어버려서 눈물날 것 같아 >> 입력하신 내용에서 슬픔이 느껴집니다.
 내용 : 나 핸드폰 잃어버려서 개박치네 >> 입력하신 내용에서 슬픔이 느껴집니다.
 내용 : 우와 무지개다 대박 >> 입력하신 내용에서 놀람이 느껴집니다.
 내용 : 나 최우수상 탔어! >> 입력하신 내용에서 행복이 느껴집니다.
 내용 : 고양이 귀여워 예쁘다 >> 입력하신 내용에서 놀람이 느껴집니다.

최종프로젝트 | 구현 코드

담당 : 윤수영

KoBert 를 활용한 문장 내 감정 인식

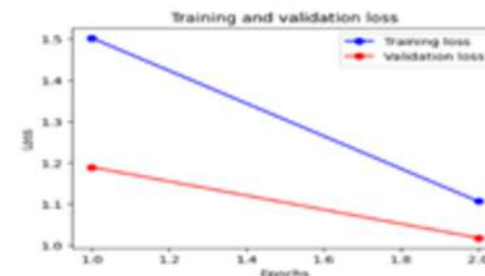
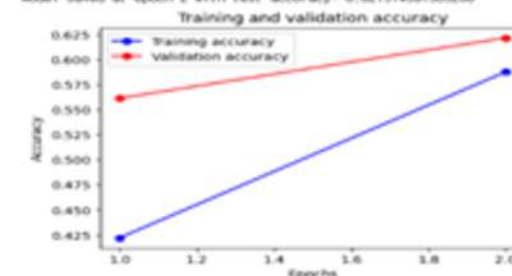
- 파라미터 테스트 (Max Length, Batch Size, Learning Rate 변경하여 확인)

```
max_len = 128
batch_size = 128
learning_rate = 5e-5
num_epochs = 10
max_grad_norm = 1.0
log_interval = 200
warmup_ratio = 0.1
```

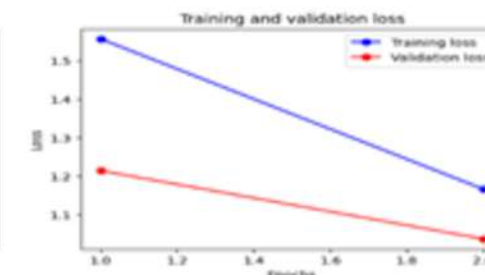
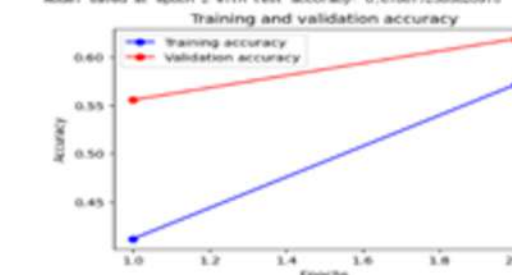
```
max_len = 256
batch_size = 128
learning_rate = 2e-5
num_epochs = 10
max_grad_norm = 1.0
log_interval = 200
warmup_ratio = 0.1
```

```
max_len = 256
batch_size = 128
learning_rate = 2e-5
num_epochs = 10
max_grad_norm = 1.0
log_interval = 200
warmup_ratio = 0.1
```

```
Training Epoch 2: 100% 80% 60% 40% 20% 0% 0.000000 1.44%
epoch 2 batch id 1 loss 1.183971643447076 train acc 0.5546976
epoch 2 batch id 201 loss 1.2136420011520096 train acc 0.5691079270640766
epoch 2 batch id 401 loss 1.1979949670791626 train acc 0.5804434226932669
epoch 2 batch id 601 loss 0.9487634897232056 train acc 0.5875489968396023
epoch 2 train acc 0.587630849738946 train loss 1.1006470267541895
Evaluating Epoch 2: 100% 15% 10% 5% 0% 0.000000 5.88%
epoch 2 test acc 0.621374591503258 test loss 1.0178004456501362
GPU 메모리 사용량: 3.21 GB
조기 종료
Model saved at epoch 2 with test accuracy: 0.621374591503258
```



```
Training Epoch 2: 100% 80% 60% 40% 20% 0% 0.000000 1.28%
epoch 2 batch id 1 loss 1.1279079914093018 train acc 0.5709125
epoch 2 batch id 201 loss 1.213642145791802 train acc 0.5559312810940234
epoch 2 batch id 401 loss 1.0073088339614868 train acc 0.5800324109502195
epoch 2 batch id 601 loss 1.153060092163006 train acc 0.5709104617304492
epoch 2 train acc 0.571390060206966 train loss 1.1664036915156536
Evaluating Epoch 2: 100% 15% 10% 5% 0% 0.000000 2.90%
epoch 2 test acc 0.6186172385620915 test loss 1.037614889432044
GPU 메모리 사용량: 3.22 GB
조기 종료
Model saved at epoch 2 with test accuracy: 0.6186172385620915
```



KoBERT 모델과 토큰라이저 로드 완료
 /usr/local/lib/python3.10/dist-packages/transformers/optimization.py:591: FutureWarning: This implementation of AdamW is deprecated.
 warnings.warn()

Training Epoch 1: 0% 0/609 [00:01<7, 715/s]

```
OutOfMemoryError                                Traceback (most recent call last)
<ipython-input-56-585b0fed6735> in <cell line: 8>()
      6 log_interval = 200 # 학습 로그를 출력할 간격 설정
      7 warmup_ratio = 0.1 # 학습 초기 단계에서의 warmup 비율 설정
----> 8 learning(max_len, batch_size, warmup_ratio, num_epochs, max_grad_norm, log_interval, learning_rate, try_count)
      9
     10 try_count += 1
```

```
20 frames
/usr/local/lib/python3.10/dist-packages/transformers/activations.py in forward(self, input)
     76
     77 def forward(self, input: Tensor) -> Tensor:
--> 78     return self.act(input)
     79
     80
```

OutOfMemoryError: CUDA out of memory. Tried to allocate 768.00 MiB. GPU

```
model_name = 'test_kobert_early3'
max_len = 128
batch_size = 128
learning_rate = 5e-5
for i in range(len(list_sentence)):
    print(predict(list_sentence[i], model_name, max_len, batch_size))
```

내용 : 우와 무지개다 대박 >> 입력하신 내용에서 놀람이 느껴집니다.
 None
 내용 : 여기 조금 무섭다 >> 입력하신 내용에서 놀람이 느껴집니다.
 None
 내용 : 잔잔 예쁘다 >> 입력하신 내용에서 혐오가 느껴집니다.
 None
 내용 : 나 핸드폰 잃어버려서 눈물날 것 같아 >> 입력하신 내용에서 공포가 느껴집니다.
 None
 내용 : 나 핸드폰 잃어버려서 개박치네 >> 입력하신 내용에서 행복이 느껴집니다.
 None
 내용 : 우와 무지개다 대박 >> 입력하신 내용에서 놀람이 느껴집니다.
 None
 내용 : 나 최우수상 왔어! >> 입력하신 내용에서 공포가 느껴집니다.
 None
 내용 : 고양이 귀여워 예쁘다 >> 입력하신 내용에서 놀람이 느껴집니다.
 None

```
model_name = 'test_kobert_early4'
max_len = 256
batch_size = 128
learning_rate = 2e-5
for i in range(len(list_sentence)):
    print(predict(list_sentence[i], model_name, max_len, batch_size))
```

내용 : 우와 무지개다 대박 >> 입력하신 내용에서 혐오가 느껴집니다.
 None
 내용 : 여기 조금 무섭다 >> 입력하신 내용에서 혐오가 느껴집니다.
 None
 내용 : 잔잔 예쁘다 >> 입력하신 내용에서 공포가 느껴집니다.
 None
 내용 : 나 핸드폰 잃어버려서 눈물날 것 같아 >> 입력하신 내용에서 공포가 느껴집니다.
 None
 내용 : 나 핸드폰 잃어버려서 개박치네 >> 입력하신 내용에서 행복이 느껴집니다.
 None
 내용 : 우와 무지개다 대박 >> 입력하신 내용에서 혐오가 느껴집니다.
 None
 내용 : 나 최우수상 왔어! >> 입력하신 내용에서 행복이 느껴집니다.
 None
 내용 : 고양이 귀여워 예쁘다 >> 입력하신 내용에서 혐오가 느껴집니다.
 None

과제

OCR과 나이트베이즈로 훈련한 금치어 데이터 모델이 적용된 클린 게시판 만들기

팀장 윤수영

팀원 정준영 송지미

윤수영 작업 내용

- django - 이미지업로드, 문자식별
- 나이트 베이즈 분류기 - 스팸여부 확인
- 구글 OCR 활용한 주피터노트북 테스트
- 게시판 Back end 구현 (조회/등록/상세)
- JavaScript : axios를 활용한 CORS
- spring boot : 조회 목록 페이지징처리

과제 | 주제 및 구현 방법

과제 내용

- 나이트 베이스 훈련 데이터를 활용하여 파일 업로드 시 OCR을 활용 하고 게시판에 업로드할 수 있는지 없는지를 구별하는 기능 개발

기능 구현

- 영화 포스터 게시판에 이미지 업로드 시 스팸 이미지 분석 및 영화 포스터 이미지만 서버에 저장

기능 구현 방법

- Spring Boot와 Vue.js의 각 역할에 대한 이해
- 게시판에서 이미지를 업로드 할 시 django, Vue.js의 크로스통신 구현

과제 | 개발 환경

담당 : 윤수영

SW Version

- java:"17.0.10"2024-01-16LTS
- python:3.12.3
- node:v20.12.2
- Springboot:3.2.6
- Oracle11g

IDE

- STS4:4.22.1.RELEASE
- VisualCode:1.90
- OracleVMVirtualBox:6.1
- ubuntu:22.04.4LTS
 - pycharm:2024.1.3
 - python:3.10.14
 - dJango:5.0.4
 - googlecolab
 - Python3.10.12

Moduel

- pandas
- cv2
- pytesseract
- sklearn
 - naive_bayes:GaussianNB
 - feature_extraction.text
 - CountVectorizer
- google-cloud-vision
 - core:1.31.5
 - google-cloud-vision:2.11.1

과제 I 개발 일정

담당 : 윤수영

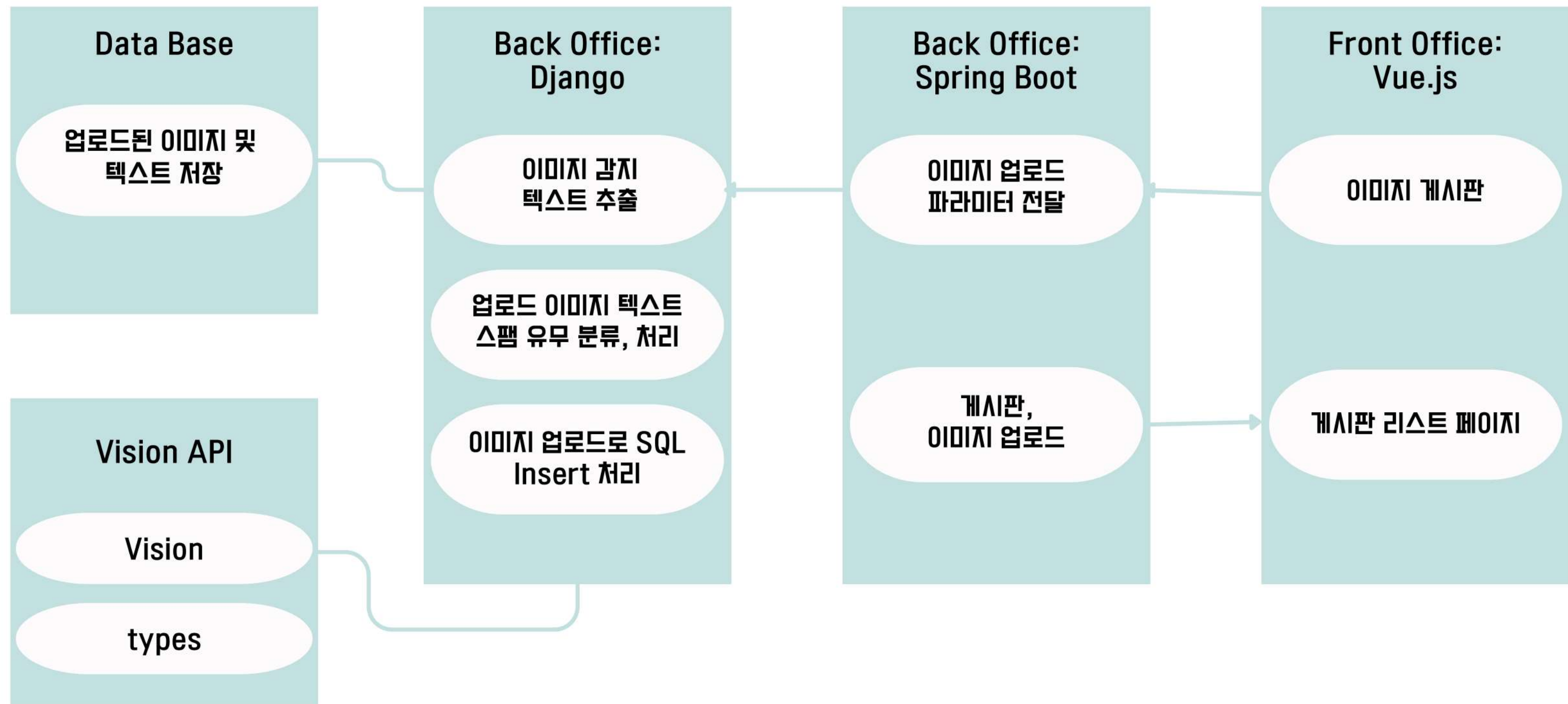
기간 : 2024.06.07 ~ 2024.06.15

- 1 2024.06.07
 - 주제 회의
 - 역할분담
 - 페이지 디자인 선정
 - API 선정 및 실행
- 2 2024.06.08
 - SpringBoot 개발 완료
 - API 선정 및 설치
- 3 2024.06.09 ~ 2024.06.10
 - Vue.js 개발(게시판,업로드)
- 4 2024.06.10 ~ 2024.06.12
 - 이미지 텍스트 내 스팸 분류 개발
 - dJango 서비스 개발

- 5 2024.06.12 ~ 2024.06.13
 - Back - Front 연결
 - 상세 기능 추가
 - 오류 수정
- 6 2024.06.12 ~ 2024.06.13
 - PDF 문서 작성
 - PPT 발표 문서 작성
 - 서비스 테스트
- 7 2024.06.18
 - 발표

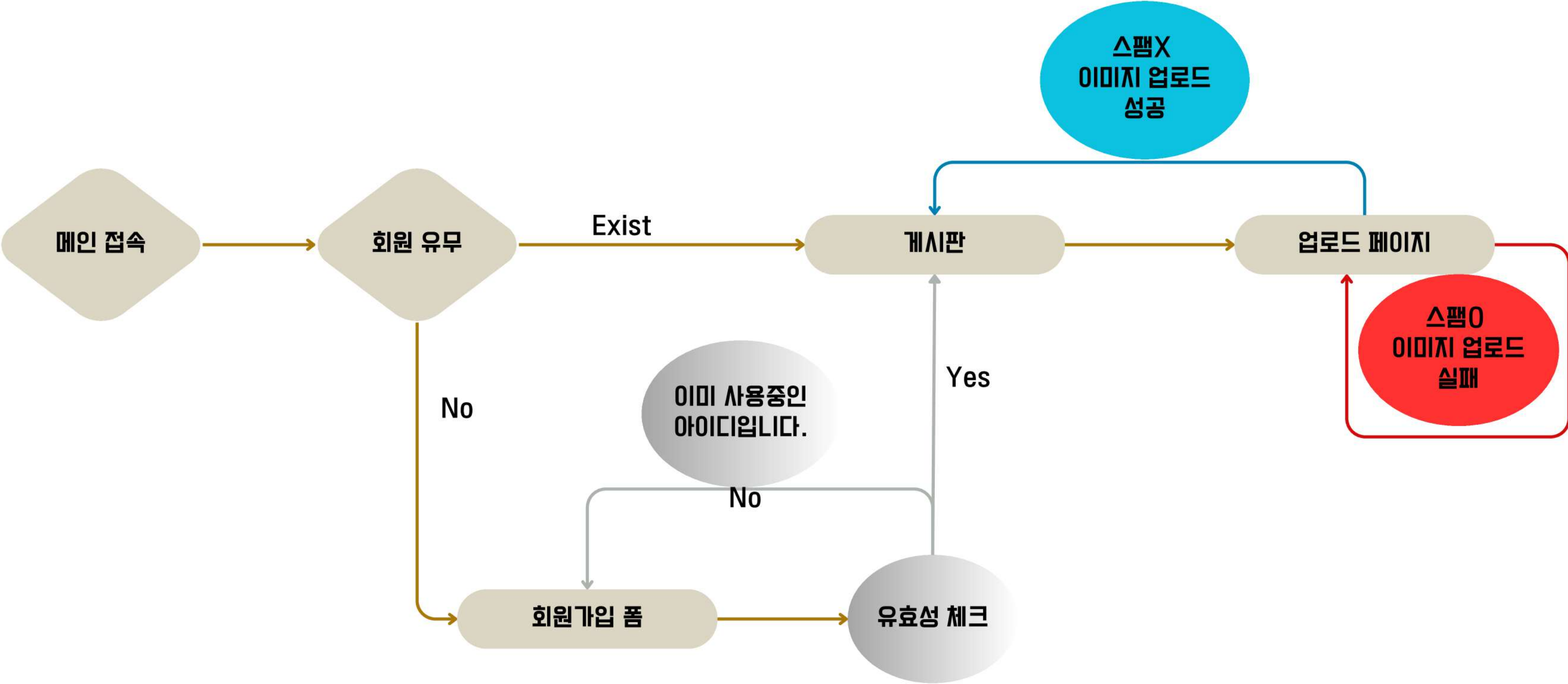
과제 | 아키텍처

담당 : 윤수영



과제 | 구조도

담당 : 윤수영

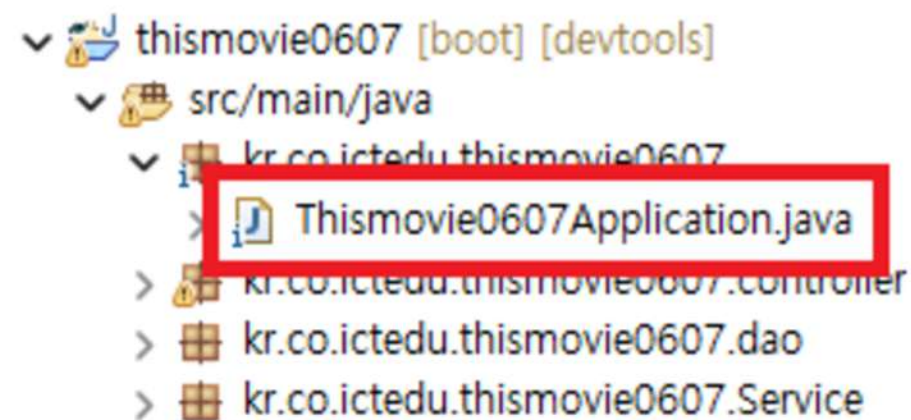


과제 | 구현 코드

담당 : 윤수영

axios를 사용한 JSON 데이터 CORS

• SpringBoot에서CORS설정



```
// 아래 경로에 있는 패키지로 import
//import org.springframework.web.servlet.config.annotation

@SpringBootApplication
public class Thismovie0607Application {

    public static void main(String[] args) {
        // 생략
        @Bean
        public WebMvcConfigurer corsConfigurer() {
            return new WebMvcConfigurer() {
                @Override
                public void addCorsMappings(CorsRegistry registry) {
                    registry.addMapping("/**")
                        .allowedOrigins("http://허용할IP:3000/")
                        .allowedHeaders("*")
                        .allowedMethods("*").maxAge(9999);
                }
            };
        }
    }
}
```

경로 : [프로젝트명]\src\main\java\[패키지경로]\[프로젝트명]Application.java

과제 | 구현 코드

담당 : 윤수영

Vue에서 CORS설정

- axios를 사용하여 SpringBoot로 데이터 전송
- axios는 기본적으로 브라우저의 CORS정책을 따름
- 게시물 등록시 SpringBoot로 formdata를 보내는 메서드

경로 : addList.vue>submitForm()

```
submitForm() {  
  if (!this.isImgDataValid) {  
    alert("유효한 이미지 데이터를 확인해주세요.");  
    return;  
  }  
  const formData = new FormData();  
  for (let i = 0; i < this.selectedFiles.length; i++) {  
    const file = this.selectedFiles[i];  
    formData.append('file', file);  
  }  
  formData.append("title", this.title);  
  formData.append("writer", this.writer);  
  formData.append("content", this.content);  
  formData.append("reip", this.reip);  
  
  axios.post(`${this.backendUrl}/uboard/upAdd`, formData, {  
    headers: { "Content-Type": "multipart/form-data" },  
  })  
    .then(() => {  
      this.fetchFiles();  
      this.$router.push({ name: 'list' });  
    })  
    .catch(error => {  
      alert(error.message);  
      alert('데이터 업로드 중 오류가 발생했습니다.');
```


과제 | 구현 코드

담당 : 윤수영

Vue에서 CORS설정

```
fetchData(num) {  
  console.log(num);  
  
  axios.get(`http://192.168.0.39/thismovie0607/uboard/upDetail/${num}`)  
    .then((resp) => {  
      this.result = resp.data;  
    })  
    .catch((err) => {  
      console.error(err);  
    });  
},  
}
```

```

```

- 게시글 리스트 조회시 SpringBoot에서 formdata를 받아오는 메서드
 - this.result에 데이터 리스트를 저장
- template>이미지 태그>src를 바인딩해서 이미지를 불러옴

과제 | 구현 코드

담당 : 윤수영

Vue에서 CORS설정

- 이미지 스팸체크시 dJango로 formdata를 보내는 메서드

```
chkImg() {  
  // .. 섹션  
  const formData = new FormData();  
  for (let i = 0; i < this.selectedFiles.length; i++) {  
    formData.append("file1", this.selectedFiles[i]);  
    formData.append("category", this.imageName);  
  }  
  axios.post('http://192.168.0.131:9000/thisisspam/insert_img'  
    , formData, {  
      headers: {  
        'Content-Type': 'multipart/form-data'  
      }  
    })  
  .then((response) => {  
    console.log(response); // JSON 데이터를 콘솔에 출력  
    this.imgdata = JSON.stringify(response.data, null, 2);  
    // ^ JSON 데이터를 문자열로 변환하여 저장  
    const imgDataParsed = JSON.parse(this.imgdata)["spam"];  
    if (imgDataParsed === 1) {  
      alert('상업적인 사진은 업로드되지 않습니다.');    }  
  })  
  .catch((error) => {  
    console.error(error);  
    alert('이미지 처리 중 오류가 발생했습니다.');  })  
},
```

경로 : addList.vue>chkImg()

이미지가 스팸인지 확인하는 게시판 과제 | 구현 코드

(1) Modelsimport정보

```
import re
import io
from google.cloud import vision
from google.cloud.vision_v1 import types
from django.db import models
import pytesseract
import cv2
import pandas as pd
from sklearn.naive_bayes import GaussianNB
from sklearn.feature_extraction.text import CountVectorizer
import numpy as np
import os

# Create your models here.
#정보와 경고 로그를 제외하고 오류 로그만 표시 - 필요 없는 로그 줄이고
#필요한 것만 보기 위함
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'
os.environ['GOOGLE_APPLICATION_CREDENTIALS'] = '#####'
```


이미지가 스팸인지 확인하는 게시판 과제 | 구현 코드

담당 : 윤수영

(1) Modelsimport정보

- 기본 패키지를 제외한 나머지 패키지는 pip install을 진행하고 import .

```
from google.cloud import vision
from google.cloud.vision_v1 import types
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'
os.environ['GOOGLE_APPLICATION_CREDENTIALS'] = '#####'
```

- google cloud vision은 google cloud가입후 개발환경에서 설치 진행
 - window(googlecolab),linux(django) OS모두 사용 가능
 - 가입: <https://cloud.google.com/vision?hl=ko>
 - 설치: <https://cloud.google.com/vision/docs/libraries?hl=ko>

이미지가 스팸인지 확인하는 게시판 과제 | 구현 코드

담당 : 윤수영

(2)insert_img():이미지를 받아 스팸인지 아닌지 판별하는 Views

- POST 방식으로 파일을 받아와서 UPLOAD_DIR 폴더에 file_name으로 저장
 - if문을 사용해서 file1 요청이 파일에 포함되었는지 확인
- 파일을 저장할 경로에 'wb+'쓰기 모드로 open
- 파일 내용을 chunk 단위로 읽어서 저장.

```
@csrf_exempt
def insert_img(request):
    if request.method == 'POST' and 'file1' in request.FILES:
        file = request.FILES['file1']
        file_name = file.name
        file_path = os.path.join(UPLOAD_DIR, file_name)

        with open(file_path, 'wb+') as destination:
            for chunk in file.chunks():
                destination.write(chunk)
```

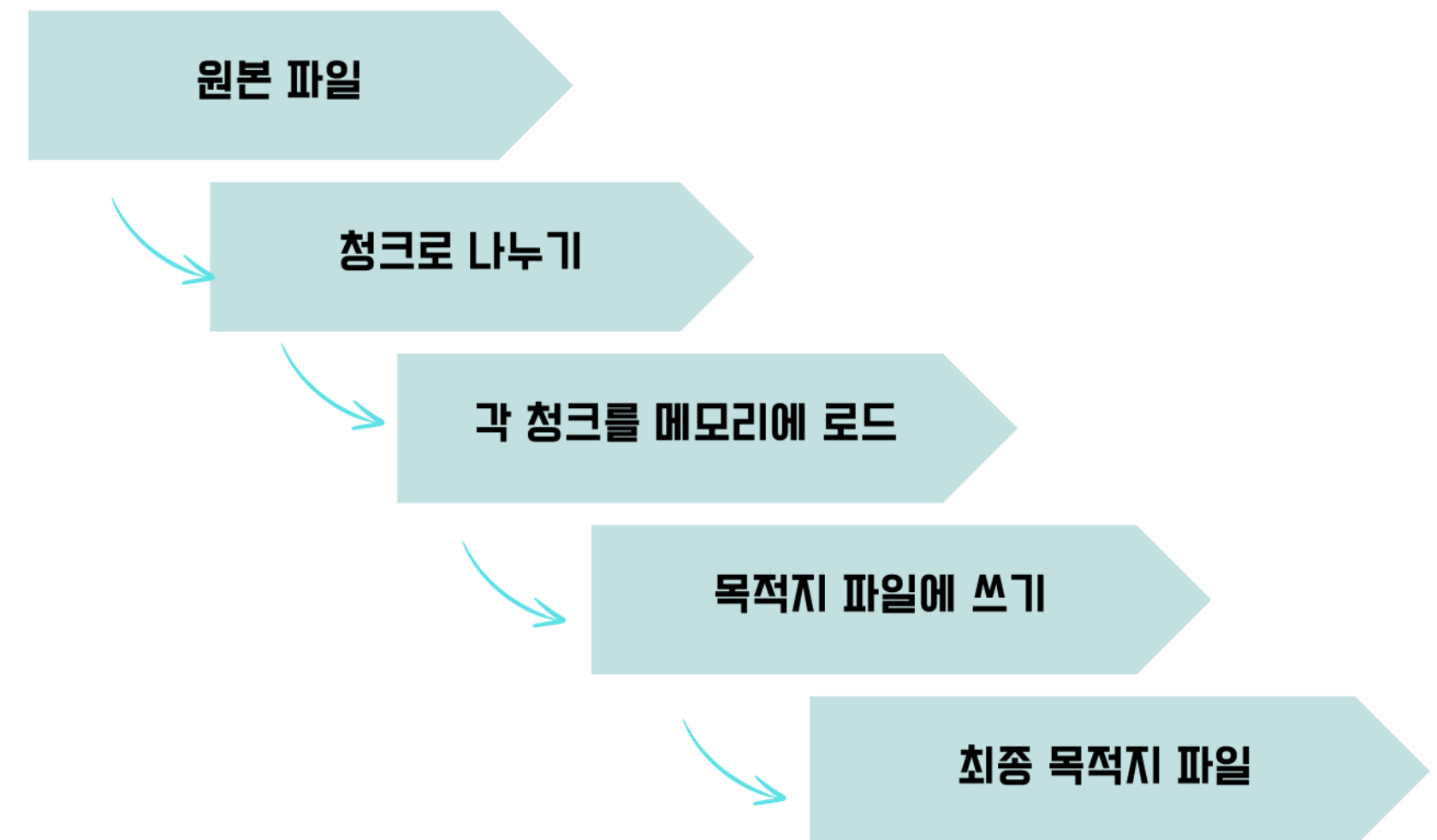
이미지가 스팸인지 확인하는 게시판 과제 | 구현 코드

담당 : 윤수영

파일 내용을 chunk 단위로 읽어서 저장 ?

- 대용량 파일을 효율적으로 처리할 수 있음
- `file.chunks()` 메서드
- Django의 `UploadedFile` 객체에서 제공하는 메서드
- 파일을 작은 청크(조각) 단위로 반환
- 기본 청크 크기는 Django default 설정값으로 되어있음
- `FILE_UPLOAD_MAX_MEMORY_SIZE = 2.5MB`
- `chunks()` 메서드 청크 크기 조정 가능

chunk 방식 파일 처리 과정 흐름도



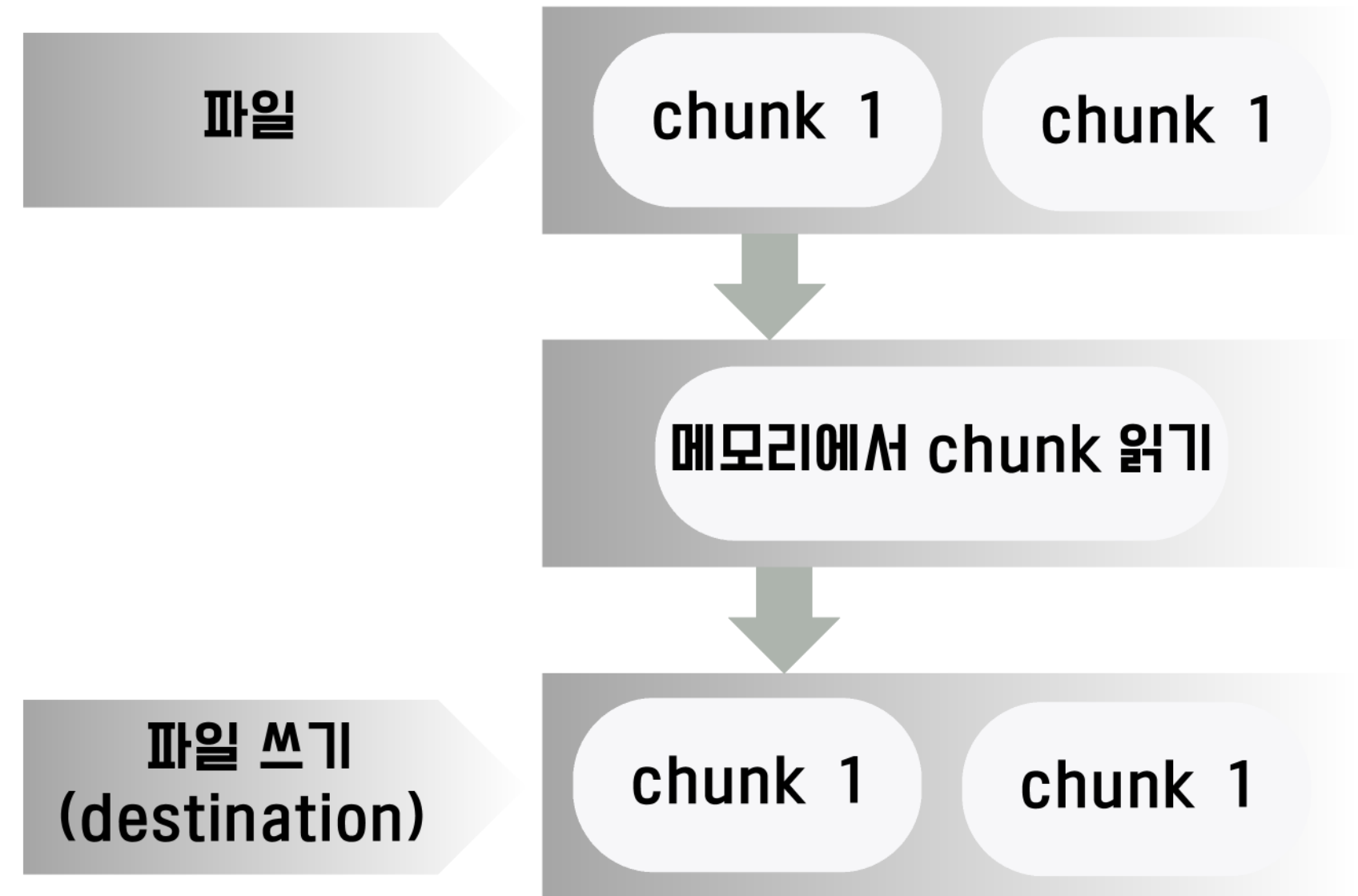
이미지가 스팸인지 확인하는 게시판 과제 | 구현 코드

담당 : 윤수영

청크 단위로 파일 읽고 쓰기

- with open(file_path, 'wb+') as destination:
 - 파일을 쓰기 모드 (wb+)로 열고
 - destination이라는 파일 객체 생성
- for chunk in file.chunks():
 - chunks() 메서드를 사용하여 파일을 청크 단위로 읽음
 - 각 청크는 메모리에 한 번에 로드됨
 - 전체 파일을 한꺼번에 메모리에 로드하는 것보다 효율적
- destination.write(chunk):
 - 각 청크를 목적지 파일에 write
 - 파일 write가 완료될 때까지 반복

chunk 방식 파일 처리 과정



- 전체 파일이 여러 청크로 나누어져 있음.
- 각 청크는 메모리에 한 번에 로드됨.
- 각 청크를 목적지 파일에 씀.

이미지가 스팸인지 확인하는 게시판 과제 | 구현 코드

담당 : 윤수영

(2)insert_img():이미지를 받아 스팸인지 아닌지 판별하는 Views

```
// .. 이어서  
myModel = MyResNetSpamModel()  
  
text2 = myModel.detect_texts(file_path)  
spam = myModel.predict_spam(text2)  
  
resJson = {"spam":spam}  
print(f"resJson => {resJson}")  
return JsonResponse(resJson)  
else:  
    return JsonResponse({"error": "Invalid request"}, status=400)
```

- detect_texts(file_path)
 - file_path를 매개변수로한 이미지에서 텍스트를 추출하는 메서드
- predict_spam(text2)
 - detect_texts()에서 추출한 텍스트 데이터를
 - GaussianNB으로 학습된 데이터에 상업적인 단어가 들어갔는지 판별하는 메서드
- Vue로보내는Json데이터
 - predict_spam()에서 반환된 값을 JSON형식으로 반환함.

이미지가 스팸인지 확인하는 게시판 과제 | 구현 코드

담당 : 윤수영

(3)isThisSpam():pytesseract를 이용한 이미지 텍스트 전처리 모델

```
def isThisSpam(self, imgName):  
    img = cv2.imread(imgName)  
    imageTarget = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)  
    text = pytesseract.image_to_string(imageTarget, lang='kor+eng')  
    cleaned_content = re.sub('[^가-힣ㄱ-ㅎㅏ-ㅣ|a-zA-Z0-9]', ' ', text)  
    return cleaned_content
```

- cv2.imread()로 이미지를 불러와서 cv.cvtColor()메서드로 BGR형식의 이미지를 RGB로 변환.
pytesseract.image_to_string(imageTarget, lang='kor+eng')
 - pytesseract라이브러리로이미지에서텍스트추출
 - imageTarget:텍스트를추출할이미지
 - lang='kor+eng'인식할언어
- re.sub('[^가- ㄱ-ㅎㅏ-ㅣ|a-zA-Z0-9]','',text)
 - 정규표현식을 사용해서 한글(완성형및자모),영어대소문자, 숫자를 제외한 모든 문자를 공백으로 치환

이미지가 스팸인지 확인하는 게시판 과제 | 구현 코드

담당 : 윤수영

isThisSpam()메서드를 사용하지 않은 이유

pytesseract 모델은
배경이 복잡할수록
문자 인식률이 떨어지며,
특히 변형된 폰트의
한국어 인식이 안됨.



ex1) 폰트정확, 영어이미지

```
// 20240607203025
// http://192.168.0.131:9000/thisisspam/yes_

{
  "res": "web_site",
  "probability": "0.73",
  "text": "RYAN GOSLING EMMA STONE A LA LA
```



ex2) 폰트정확, 한글이미지

```
// 20240607203121
// http://192.168.0.131:9000/thisisspam/

{
  "res": "web_site",
  "probability": "0.98",
  "text": "비상선언 "
```



ex3) 폰트변형, 한글이미지

```
20240607203324
http://192.168.0.131:9000/thisisspam/yes_

{
  "res": "torch",
  "probability": "0.28",
  "text": "Es | 3 5 2
          4 coming soon
```


이미지가 스팸인지 확인하는 게시판 과제 | 구현 코드

담당 : 윤수영

(3)detect_texts(): Google API를 이용한 이미지 텍스트 전처리 모델

```
def detect_texts(self, imgName):
    # Vision API 클라이언트 설정
    client = vision.ImageAnnotatorClient()

    # 이미지 읽기
    with io.open(imgName, 'rb') as image_file:
        content = image_file.read()

    image = types.Image(content=content)

    # 텍스트 감지
    response = client.text_detection(image=image)
    texts = response.text_annotations

    # 감지된 텍스트 추출
    text = [re.sub('[^가-힣ㄱ-ㅎㅏ-ㅣ|a-zA-Z0-9]', ' ', text.description) for text in texts]

    return text
```

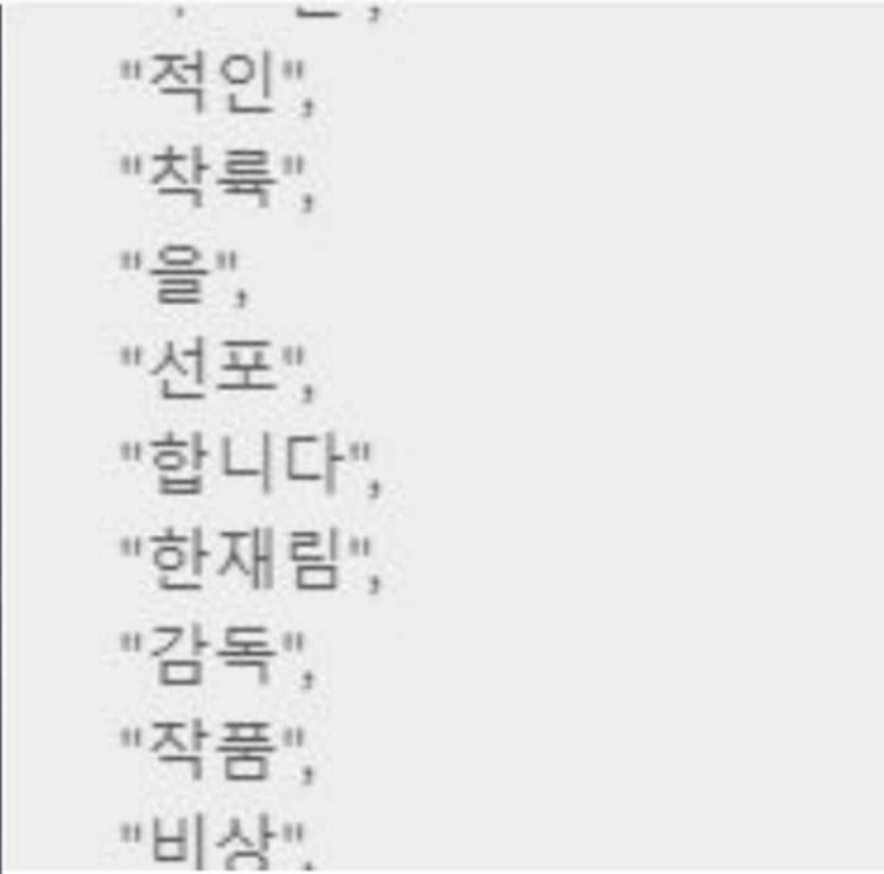
- vision.ImageAnnotatorClient(): Vision API 클라이언트 설정
 - os.environ['TF_CPP_MIN_LOG_LEVEL']='2'
 - os.environ['GOOGLE_APPLICATION_CREDENTIALS']='인증키 경로'
- io 모듈로 이미지를 읽어 Google Cloud Vision 모듈인 types로 이미지를 저장.
- text_detection()을 사용하여 텍스트를 추출한 후 모든 정보를 response에 저장.
- response데이터에서 text_annotations 데이터를 tests에 저장.
 - text_annotations외에 labels등 다양한 정보 있음.

- 정규표현식을 사용하여 추출한 문자들을 전처리하여 text로 반환 .
- re.sub('[^가- ㅏ-ㅎㅏ-ㅣ|a-zA-Z0-9]', '', text)
 - 정규표현식을 사용해서 한글(완성형및자모),영어대소문자, 숫자를 제외한 모든 문자를 공백으로 치환

이미지가 스팸인지 확인하는 게시판 과제 | 구현 코드

담당 : 윤수영

(3)detect_texts(): Google API를 이용한 이미지 텍스트 전처리 모델



ex) 폰트정확, 한글이미지

구분 가능한 모든 문자 추출



```
// 20240614211157
// http://192.168.0.131:9000/thisisspam/insert_img
```

```
{
  "spam": 1
}
```



```
// 20240614211324
// http://192.168.0.131:9000/thisisspam/insert_img
```

```
{
  "spam": 0
}
```


이미지가 스팸인지 확인하는 게시판 과제 | 구현 코드

담당 : 윤수영

(4)predict_spam(): GaussianNB를 이용한 스팸 문자 판별

- 학습시킬 데이터셋 생성
 - data={'index':range(x,y),'document':[],'label':[]}
 - email:정상문장/단어,스팸문장/단어
 - status:nospam(정상),spam(스팸)
- 데이터셋을 DataFrame으로 변환 후 단어 벡터화
 - CountVectorizer()
- 가우시안 나이브베이즈 선언 및 벡터화 한 데이터셋 학습

```
# 데이터를 데이터프레임으로
df = pd.DataFrame(data)

# 단어벡터화
vectorizer = CountVectorizer()

# X : 'email' 컬럼을 특징 벡터로 변환
X = vectorizer.fit_transform(df['email']).toarray()
# print(f"{df['email'][0]}, {X[0]}")

# Y : 'status' 컬럼을 숫자로 변환 nospam 0 , spam 1
y = df['status'].map({'nospam': 0, 'spam': 1})

# 가우시안 나이브 베이즈 객체를 생성
clf = GaussianNB()
clf.fit(X, y)
# msg = {0: '스팸 아님', 1: '스팸'}
```

이미지가 스팸인지 확인하는 게시판 과제 | 구현 코드

담당 : 윤수영

(4)predict_spam(): GaussianNB를 이용한 스팸 문자 판별

- 데이터 학습 및 예측: GaussianNB
 - GaussianNB는 연속적인 값을 가진 특징(예:숫자형)을 처리하는데 주로 사용
 - 텍스트 데이터는 문자열로 되어 있어 바로 사용할 수 없음
 - 문자열 데이터를 모델이 이해할 수 있는 벡터 형태로 변환하는게 벡터화
- CountVectorizer: 각문서를 단어의 빈도수로 표현
 - 많이 출현할수록 빈도수가 높아짐.
 - GaussianNB로학습(fit.())

이미지가 스팸인지 확인하는 게시판 과제 | 구현 코드

담당 : 윤수영

(4)predict_spam(): GaussianNB를 이용한 스팸 문자 판별

1.단어 빈도 계산:

- a. "할인"단어가 spam 라벨 문서에서 30번, nospam 라벨 문서에서 10번 등장하면
- b.이 단어의 빈도는 클래스 별로 다르게 기록

2.조건부 확률 계산: 각 클래스에 대해 조건부 확률 계산 => 특정 단어가 주어진 클래스에서 등장할 확률

- a. "할인" 단어가 spam라벨 문서에서 자주 등장하면, 조건부 확률 증가

3.사후 확률 계산:

- a.새로운 문서에 대해 각 클래스의 사후 확률 계산
- b.각 단어의 조건부 확률을 곱하고 클래스의 사전 확률을 곱하여 새문서가 특정 클래스에 속할 확률 계산

이미지가 스팸인지 확인하는 게시판 과제 | 구현 코드

담당 : 윤수영

(4)predict_spam(): GaussianNB를 이용한 스팸 문자 판별

- score:텍스트의 예측 확률 저장하는데 사용, label길이의 1차원 배열
- for문으로 이미지에서 추출한 단어 리스트를 하나씩 순회하며 벡터화>예측>결과
 - 벡터화:test_X=vectorizer.transform([t]).toarray()
 - 예측:GaussianNB.predict_proba(test_X)
 - 결과(확률분포):predicted_prob[0]
 - 누적:score+=predicted_prob[0]
- 스팸 여부 결정
 - score[0]=스팸 아님,score[1]=스팸
 - if문에서score[1]>score[0]보다 크면 1반환, 그외는 0반환

```
resJson = {"spam":spam}
```

- 이 반환값이 views.py에서 Json형식으로 Vue로 전송된다.

```
# 예측
score = np.zeros(2)

for t in text:
    test_X = vectorizer.transform([t]).toarray()
    predicted_prob = clf.predict_proba(test_X)
    print(f"predicted_prob[0] => {predicted_prob[0]}")
    score += predicted_prob[0]
print(text)

print(f"score[0] => {score[0]}")
print(f"score[1] => {score[1]}")

# 스팸 여부 결정
if score[1] > score[0]:
    return 1 # 스팸
else:
    return 0 # 스팸 아님
```

이미지가 스팸인지 확인하는 게시판 과제 | 구현 코드

담당 : 윤수영

(4)predict_spam(): GaussianNB를 이용한 스팸 문자 판별

Movie Poster Upload

Warning
Commercial content will not be uploaded.

Image Preview



Title
글 제목

Writer
작성자

Content
내용입력

```
▶ {data: {...}, stu  
imgdata{  
  "spam": 0  
}
```

Check Img >

Movie Poster Upload

Warning
Commercial content will not be uploaded.

Image Preview



Title
글 제목

Writer
작성자

Content
내용입력

```
▶ {data: {...}, stu  
imgdata{  
  "spam": 1  
}
```

Check Img >

이미지가 스팸인지 확인하는 게시판 과제 | 구현 코드

담당 : 윤수영

최종프로젝트에 활용 : 문의 등록 화면

문의하기

문의 등록시 유의사항

(☹_☹_☹_☹) 클린봇이 상업적인 사진, 욕설, 음란성 문구를 AI 기술로 감지하고 있습니다.

test1

2024-08-27

카테고리를 선택해주세요

제목을 입력해주세요.

첨부파일

파일 선택

선택된 파일 없음

등록하기

목록으로

```
add() {  
  const formData = new FormData();  
  formData.append('qna', JSON.stringify(this.qna));  
  if (this.image) {  
    formData.append('file', this.image);  
  }  
  
  axios.post(`${process.env.VUE_APP_BACK_END_URL}/qna/qnaadd`, formData, {  
    headers: {  
      'Content-Type': 'multipart/form-data'  
    }  
  })  
  .then(response => {  
    this.qna = response.data;  
    this.$router.push(`/Qnalist`);  
  })  
}
```

- this.qna 객체를 JSON문자열로 변환한 후, formData에 file이라는 이름으로 이미지를 추가
- .then 콜백함수의 response는 정상적으로 this.qna에 담아냄

이미지가 스팸인지 확인하는 게시판 과제 | 구현 코드

담당 : 윤수영

최종프로젝트에 활용 : 문의 등록 시 이미지 처리

첨부파일

파일 선택 상담사.png

안녕하세요 오늘은 중요한 수업이 있습니다.

등록하기

목록으로



1fee58d5-91d7-4783-affe-8b5fc8fd8dc6.png



62a62d0a-e528-4fc4-8879-184a4878000f.png



94663ae5-bb73-4f35-b6bb-0305b9ae765a.png



a4a56d60-22e7-4582-a0af-dc65b3512fc2.png



a085f84e-c3a1-4b76-9d51-d14d042d3239.png



aeb525bd-bd40-438e-bc7d-0d5009684356.jpg



bfc01158-e3fe-43c7-91b3-94fa411cbf17.jpg



c15743be-a856-4ff4-821f-9e38de589421.png



f7950c2e-f7c0-411c-8f82-b55a710dbf94.png



fb94bac-4cd6-43cb-af06-1ce00e8a1bf8.jpg

- 이미지를 선택하고 이미지 데이터를 저장
- 이미지 인식 실패하면 에러 alert를 표시



```
onFileChange(e) {
  const formData = new FormData();
  this.file = e.target.files[0];
  formData.append("file1", this.file);
  axios.post(`http://192.168.0.92:9000/thisisspam/insert_img`, formData, {
    headers: {
      'Content-Type': 'multipart/form-data'
    }
  })
  .then((response) => {
    this.image = this.file;
    if (this.file) {
      this.imagePreview = URL.createObjectURL(this.file);
    }
    console.log(response.data);
    this.imgdata = JSON.stringify(response.data, null, 2);
    this.imgname = e.target.files[0];
  })
  .catch((error) => {
    console.error(error);
    alert('이미지 처리 중 오류가 발생했습니다.');
```

이미지가 스팸인지 확인하는 게시판 과제 | 구현 코드

담당 : 윤수영

최종프로젝트에 활용 : 문의 등록 시 이미지 처리

첨부파일

파일 선택 스팸이미지.png



192.168.0.56:3000 내용:
이미지 처리 중 오류가 발생했습니다.

확인

등록하기

목록으로

```
file = request.FILES['file']
content = file.read()
image = vision.Image(content=content)
```

client 모델을 사용해 이미지에서 텍스트가 있는 영역 감지

```
response = client.text_detection(image=image)
texts = response.text_annotations
filtered_texts = []
for text in texts:
    cleaned_text = re.sub('[^가-힣-ㅇㅣ-|0-9]', '', text.description)
    cleaned_text = cleaned_text.replace(' ', '') # 모든 공백 제거
    if cleaned_text: # 빈 문자열이 아닌 경우만 추가
        filtered_texts.append(cleaned_text)
return filtered_texts
```

response.text_annotations는 감지된 텍스트와 주석을 포함해 디지털 형식으로 변환

re.sub('[^가- ㅏ-하-ㅣa-zA-Z0-9]', '', text)

-정규표현식을 사용해서 한글(완성형및자모),영어대소문자,숫자를 제외한 모든 문자를 공백으로 치환

이미지가 스팸인지 확인하는 게시판 과제 | 구현 코드

담당 : 윤수영

최종프로젝트에 활용 : 문의 등록 시 이미지 처리

첨부파일

파일 선택 스팸이미지.png



```
def __init__(self):
    self.client = vision.ImageAnnotatorClient()
    self.vectorizer = CountVectorizer()
    self.model = GaussianNB()

def Project2Model(self, texts, labels):
    """텍스트와 레이블을 사용하여 모델을 학습합니다."""
    X = self.vectorizer.fit_transform(texts).toarray()
    y = labels
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
    self.model.fit(X_train, y_train)
    y_pred = self.model.predict(X_test)

myModel = Project2Model()
try:
    texts = myModel.detect_texts(file_path)
    print(f"Detected texts => {texts}")
    return JsonResponse(resJson)
except Exception as e:
    return JsonResponse({"error": f"텍스트 감지 중 오류 발생: {str(e)}"}, status=400)
if len(texts) < 3:
    return JsonResponse({"error": "텍스트 감지 실패"}, status=400)
```

- 가우시안나이브베이지 초기화 및 모델 훈련 및 학습
- 모델 사용해서 텍스트 감지 Json Response 타입으로 반환
- file_path를 매개변수로 한 이미지에서 텍스트를 추출하여 반환

중간프로젝트

AI기반 이미지 처리 머신러닝 & 딥러닝 프로젝트

팀장 윤수영

팀원 박성호 허호준 정준영 김건우 송지미 이승희 이지영

윤수영 작업 내용

- OCR
 - 주제선정, 모델구현 및 적용
- UI
 - Market : Mypage, Admin
- SERVICES
 - 마이페이지 서비스(JPA)
 - 어드민(Mybatis) 구현
- DB 설계

중간프로젝트 | 기획

담당 : 윤수영

페르소나



회원가입 어려움



OCR 회원가입



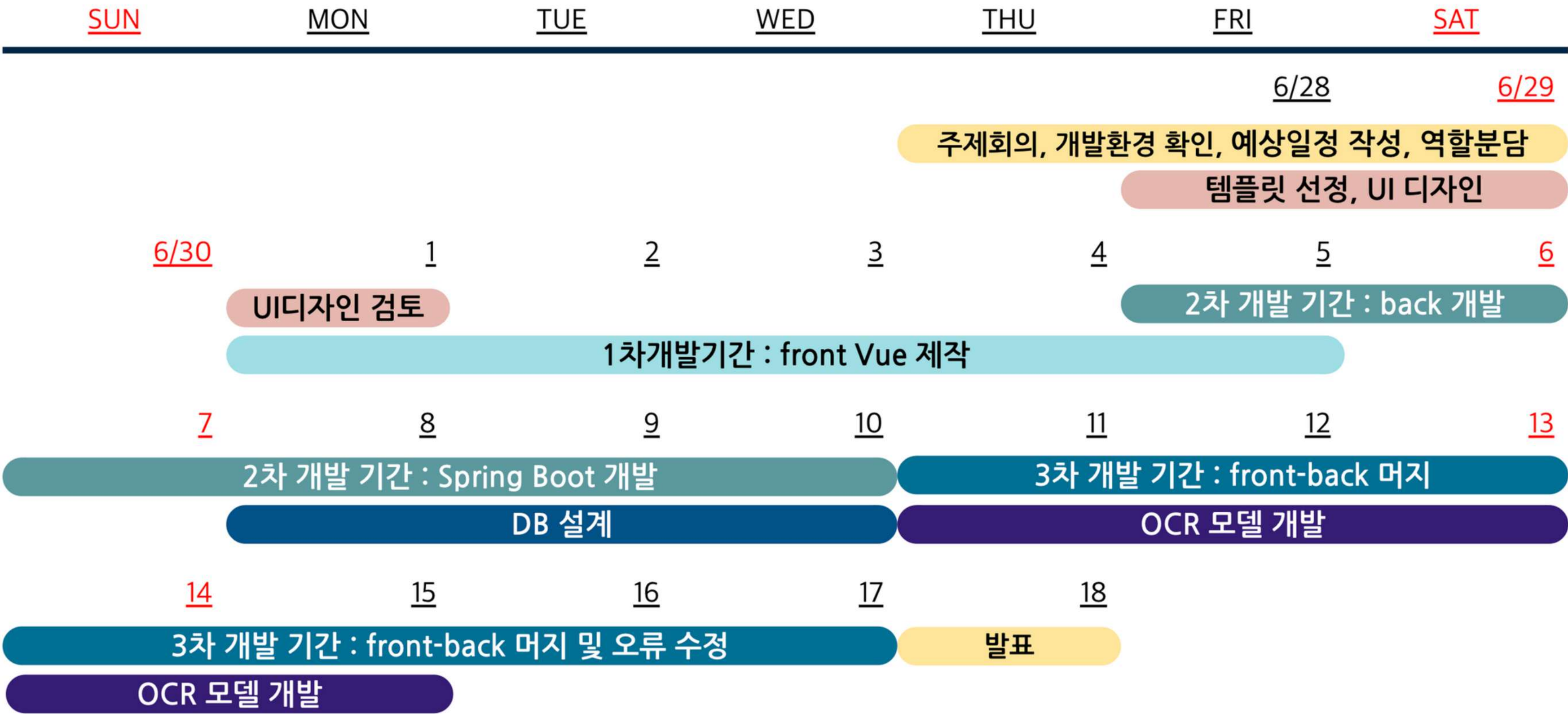
부산시 기장에 사는 70세 김숙자 할머니는 몸이 아파 장을 보지 못해서 온라인 쇼핑몰을 이용하려고 한다.
회원이입을 시도하는 것조차 불편한 김숙자 할머니에게 신분증 OCR이 편리하게 쇼핑몰을 사용할 수 있는 도움이 되었다.

쇼핑몰 사용자의 과반수가 40대 이상으로 회원가입을 할때부터 어려움을 겪는 경우가 많아
사진을 이용한 신분증 인식을 통해 보다 쉬운 회원가입을 만들겠다는 목표를 가짐

은행, 쇼핑몰 등 다양한 웹/앱에서 사용되는 신분증 OCR 인식을 벤치마킹
쇼핑몰 : 마켓컬리, OCR : 신한, 국민은행, 네이버페이

중간프로젝트 | 개발 일정

담당 : 윤수영



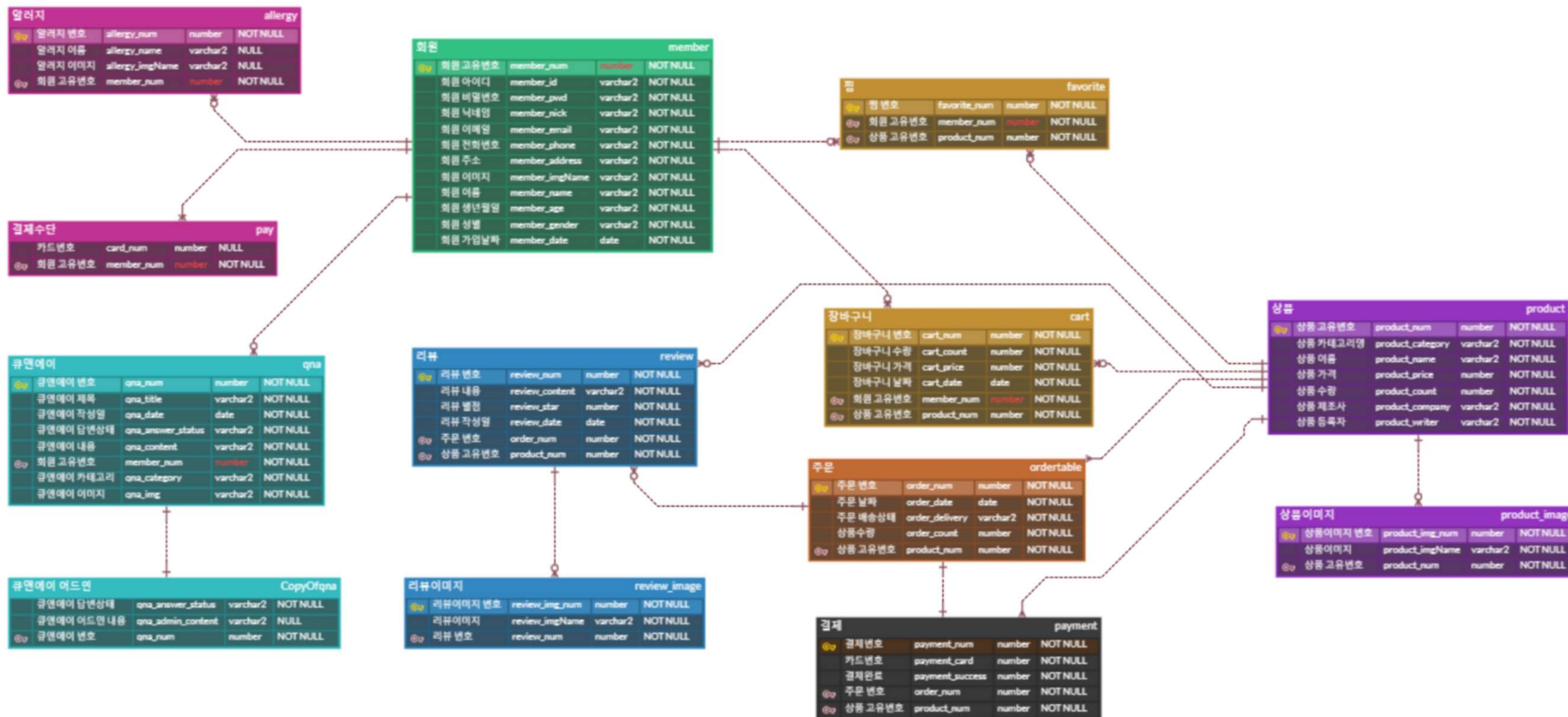
중간프로젝트 | 개발 환경

담당 : 윤수영

- IDE
 - Visual Code
 - Pycharm
 - SQL Gate
 - Jupyter Notebook
- Server
 - Apache Tomcat
 - django
 - Docker
 - ubuntu
- Front Office
 - Html 5, Css 3, JavaScript Ecma 6
 - Vue.js
- Back Office
 - Oracle 11g
 - Spring Boot
 - Java 17
 - Gradle
 - ANACONDA
 - Lombok
 - AJAX
 - jQuery
 - python
 - pandas
 - Numpy
 - Google Cloud Vision API

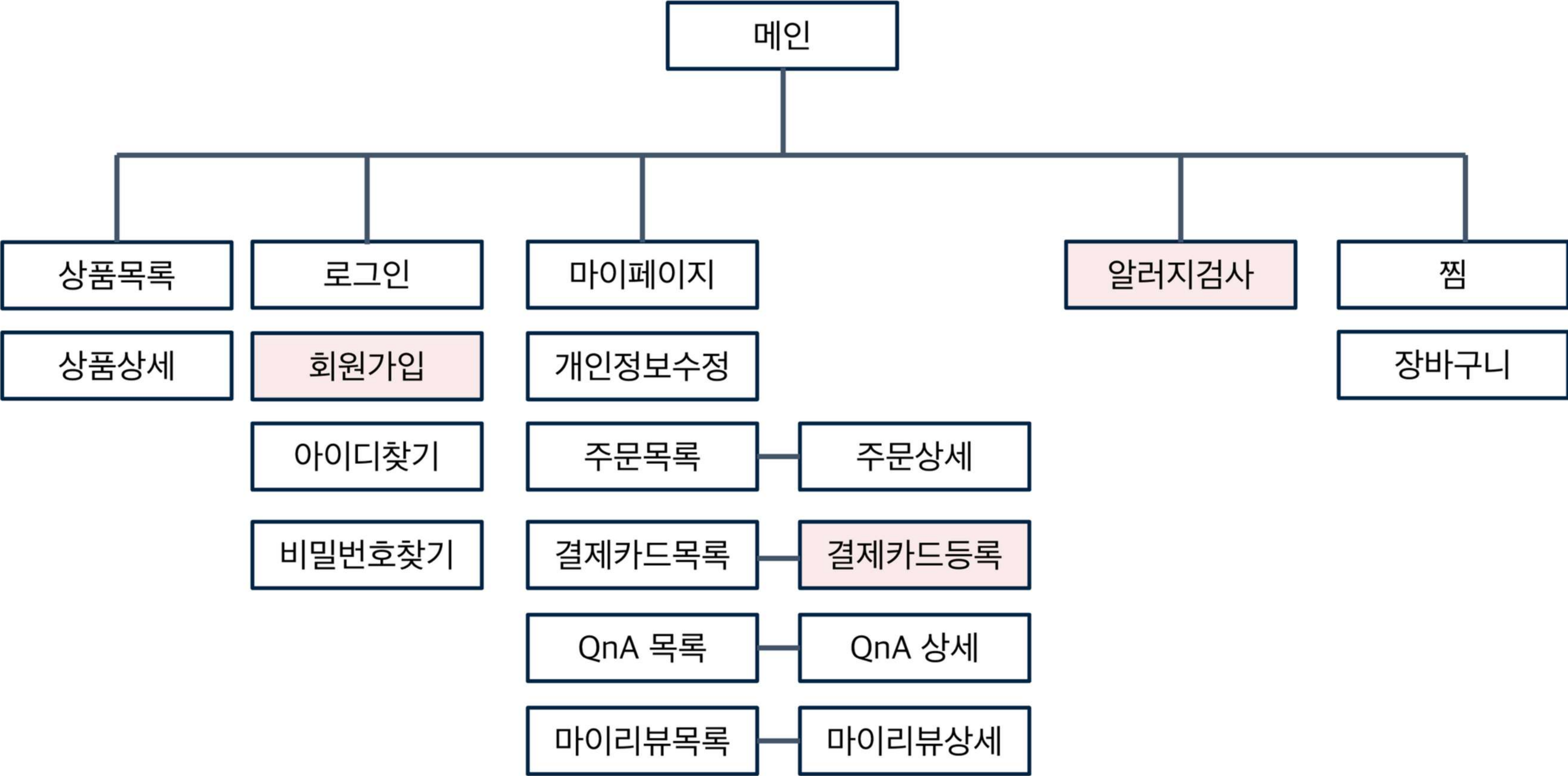
중간프로젝트 | ERD

담당 : 윤수영



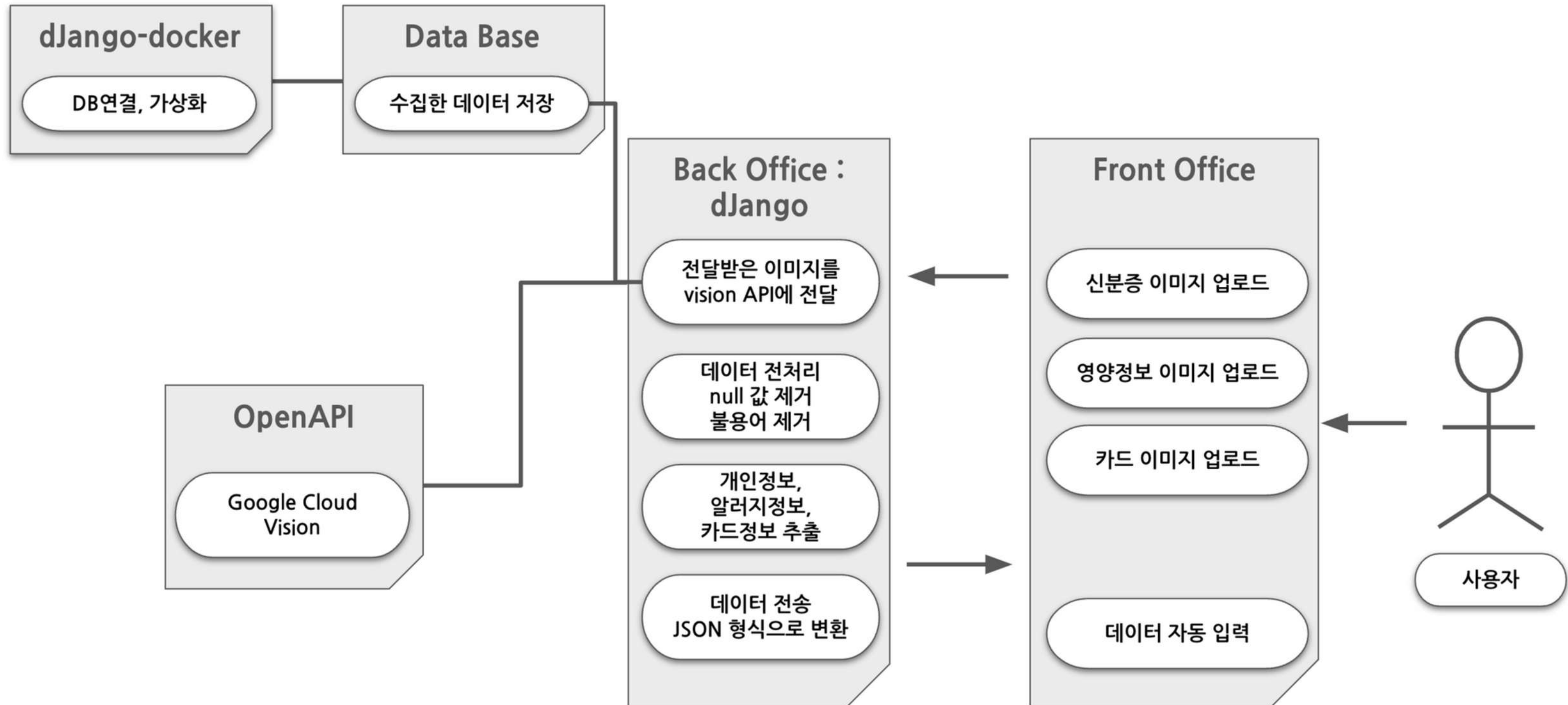
중간프로젝트 | 화면구성도

담당 : 윤수영



중간프로젝트 | 아키텍처

담당 : 윤수영



중간프로젝트 | 구현 코드

담당 : 윤수영

신분증 데이터 생성 - 1

- 실제 주민등록증을 수집할 수 없으므로 직접 생성하였습니다.
- 텍스트가 지워진 주민등록증 이미지에 샘플 데이터를 작성함
 - TextRecognitionDataGenerator 오픈소스 사용
- 주소데이터셋
 - <https://www.data.go.kr/data/15058086/openapi.do>
- 신분증 정보 생성 방법
 - 주민등록번호 랜덤함수
 - 이름(한글) : ChatGPT
 - 이름(한자) : ChatGPT
 - 주민등록번호 : 패턴 지정 및 랜덤함수
 - 주소 : 공공 데이터
 - 발급일자 : 패턴 지정 및 랜덤함수
 - 발급기관 : ChatGPT, 공공 데이터

```
# 주민번호 앞자리 생성, 발급일자 동일
data1 = {
    'yy': np.random.randint(11, 99, size=(545)),
    'mm': np.random.randint(1, 31, size=(545)),
    'dd': np.random.randint(1, 31, size=(545))
}
jumin1 = pd.DataFrame(data1)
jumin1['yy'] = jumin1['yy'].apply(lambda x: f"{x:02d}")
jumin1['mm'] = jumin1['mm'].apply(lambda x: f"{x:02d}")
jumin1['dd'] = jumin1['dd'].apply(lambda x: f"{x:02d}")
jumin1['merged1'] = jumin1['yy'] + jumin1['mm'] + jumin1['dd']
f = jumin1['merged1']

# 이름, 주소, 발급기관 csv 불러오기
name = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/prj2/idcard/data/names_three_chars.c
')
addr = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/prj2/idcard/data/병원현황(병원급).csv
')
assign = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/prj2/idcard/data/issuing_agencies.

')

# 컬럼명 변경 및 발급기관+장 추가
name.columns = ['name', 'hname']
addr = addr['소재지도로명주소']
assign['발급기관'] = assign['발급기관'] + '장'

# 개인정보 데이터 프레임 생성
combined_df = pd.DataFrame({
    'name': name['name'],
    'hname': name['hname'],
    'address': addr,
    'issuing_agency': assign['발급기관'],
    'birthdate': f,
    'additional_code': b,
    'issue_date': bday['yy'] + bday['mm'] + bday['dd']
})

# csv로 다운받기
combined_df.head()
combined_df.to_csv('/content/drive/MyDrive/Colab Notebooks/prj2/idcard/data/jumin.csv')
```


중간프로젝트 | 구현 코드

담당 : 윤수영

신분증 데이터 생성 - 2

- 오픈소스 툴인 **TextRecognitionDataGenerator**를 활용하여 다양한 폰트, 크기, 스타일의 텍스트가 포함된 이미지 생성
- 주로 학습용 데이터 생성을 위해 사용되며,
- 텍스트와 배경 이미지를 조합하여 실제 OCR 시나리오에 유사한 이미지를 생성할 때 사용함
- **다양한 폰트와 배경을 사용하여 데이터셋 다양성 확보**
 - output_dir = '결과 파일 저장 경로'
 - language = '생성 언어 선택'
 - background = '배경 이미지 사용 여부'
 - width = '생성 이미지의 너비'
 - height = '생성 이미지의 높이'
 - font_dir = '사용할 폰트 경로'
 - image_dir = '사용할 배경 이미지 경로'
- 참고 : <https://github.com/Belval/TextRecognitionDataGenerator/tree/master>

```
# 샘플로 생성한 주민등록증 정보 로드
df = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/prj2/idcard/data/jumin.csv', index_col=0)
# 폰트 파일이 저장된 경로
font_dir = "/content/drive/MyDrive/Colab Notebooks/prj2/idcard/data/font"
# 폰트 파일 목록 가져오기 (네이버 무료 폰트 50개)
font_files = [f for f in os.listdir(font_dir) if f.endswith('.ttf') or f.endswith('.otf')]

# get_text_width 함수 정의
def get_text_width(font, text):
    bbox = font.getbbox(text)
    return bbox[2] - bbox[0]

i = 0
# tqdm을 사용하여 진행 상태 표시
for i in tqdm(range(len(df))):
    try:
        # 랜덤으로 폰트 파일 선택
        selected_font = random.choice(font_files)
        selected_font_path = os.path.join(font_dir, selected_font)
        # '주민등록증' 적용 폰트
        jumin_font = ImageFont.truetype(selected_font_path, 60)
        # 개인정보 적용 폰트
        image_font = ImageFont.truetype(selected_font_path, 44)

        font_size = 44
        text_left_origin = 456 # 텍스트 시작 위치의 x 좌표
        text_top_origin = 66 # 텍스트 시작 위치의 y 좌표

        space_width = int(get_text_width(image_font, " ")) # 단어 간격 너비 계산
        space_width += 0
```


중간프로젝트 | 구현 코드

담당 : 윤수영

신분증 데이터 생성 - 3

- 각 데이터가 위치할 좌표를 x, y 축으로 설정하여 이미지 생성



```
# 데이터 프레임에서 i 번째 행 선택
row = df.iloc[i]

# 필요한 텍스트 필드 추출
name = row['name']
address = row['address']
issuing_agency = row['issuing_agency']
birthdate = str(int(row['birthdate']))
additional_code = str(int(row['additional_code']))
issue_date = str(int(row['issue_date']))

# ID 카드 이미지 로드
id_card_image_path = "/content/drive/MyDrive/Colab Notebooks/prj2/idcard/idcard.png"
id_card_image = Image.open(id_card_image_path)

# 이미지에 텍스트를 오버레이
draw = ImageDraw.Draw(id_card_image)

# 텍스트를 이미지에 추가 (좌표를 조정하여 배치 -> 데이터 1개씩 해보고 좌표 위치 조정 draw.text(x축, y축, text))
draw.text((200, 60), f"주민등록증", font=jumin_font, fill="black")
draw.text((100, 180), f"{name}", font=image_font, fill="black")
draw.text((100, 280), f"{birthdate}-{additional_code}", font=image_font, fill="black")
draw.text((100, 380), f"{address[:15]}", font=image_font, fill="black")
draw.text((100, 430), f"{address[15:30]}", font=image_font, fill="black")
draw.text((100, 480), f"{address[30:]}", font=image_font, fill="black")
draw.text((600, 580), issue_date, font=image_font, fill="black")
draw.text((300, 630), f"서울특별시 {issuing_agency}", font=jumin_font, fill="black")

# 결과 이미지 저장
output_id_card_path = f"/content/drive/MyDrive/Colab Notebooks/prj2/idcard/sampleid/id_card{i}.png"
id_card_image.save(output_id_card_path)
except Exception as e:
    print(f"Error processing row {i}: {e}")
```

중간프로젝트 | 구현 코드

담당 : 윤수영

신분증 글자 인식 1 : bounding box

- 신분증 내 텍스트를 bounding box로 위치 시각화



```
def check2(file_name):
    # Google Vision 클라이언트 생성
    client = vision.ImageAnnotatorClient()
    # 이미지 파일 읽기
    with io.open(file_name, 'rb') as image_file:
        content = image_file.read()
    image = vision.Image(content=content)
    # 텍스트 감지
    response = client.text_detection(image=image)
    texts = response.text_annotations
    # OpenCV를 사용하여 bounding box 그리기
    image = cv2.imread(file_name)
    # 작은 글자를 제외하기 위한 기준 (너비 또는 높이)
    min_width = 5
    min_height = 20
    for text in texts:
        vertices = [(vertex.x, vertex.y) for vertex in text.bounding_poly.vertices]
        # 너비와 높이 계산
        width = vertices[1][0] - vertices[0][0]
        height = vertices[2][1] - vertices[0][1]
        # 너비와 높이가 기준 이상인 경우에만 bounding box 그리기
        if width >= min_width and height >= min_height:
            cv2.polylines(image, [np.array(vertices, np.int32)], True, (0, 255, 0), 2)
    # 결과 이미지 출력
    cv2.imshow(image)
    # 에러가 발생할 경우 응답 확인
    if response.error.message:
        raise Exception(f'{response.error.message}')
```


중간프로젝트 | 구현 코드

담당 : 윤수영

신분증 글자 인식 2 : 전처리

- 이미지에서 추출한 데이터 중 필요하지 않은 문자와 기호 등을 삭제
- 전처리한 텍스트에서 '발급날짜' 가 검출되면
- 운전면허증으로 인식하게 함. (미검출시 주민등록증)

```
1 detect_texts(img_path1)
```

```
['임예준 7229172090619경기도고양시덕양구화중로 50456층화정동 20100818서울특별시강남구청장',
'임예준',
'7229172090619',
'경기도',
'고양시',
'덕양구',
'화중',
'50456',
'화정',
'20100818',
'서울',
'특별시',
'강남구',
'청장']
```

```
def detect_texts(imgName):
    """이미지에서 라벨을 감지합니다."""
    # Vision API 클라이언트 설정
    client = vision.ImageAnnotatorClient()
    # 이미지 읽기
    with io.open(imgName, 'rb') as image_file:
        content = image_file.read()
    image = types.Image(content=content)
    # 텍스트 감지
    response = client.text_detection(image=image)
    texts = response.text_annotations
    # 불필요한 텍스트 패턴
    excluded_patterns = [
        '2종보통',
        '증', '보통', '자동차', '운전', '면허증', '주민등록증'
    ]
    # 감지된 텍스트 추출 및 불필요한 텍스트 제거
    filtered_texts = []
    for text in texts:
        cleaned_text = re.sub('[^가-힣ㄱ-ㅎㅏ-ㅣ|0-9]', '', text.description)
        # "증" 앞의 숫자 제거
        cleaned_text = re.sub(r'#[d+증]', '', cleaned_text)
        for pattern in excluded_patterns:
            cleaned_text = re.sub(pattern, '', cleaned_text)
        cleaned_text = cleaned_text.replace(' ', '') # 모든 공백 제거
        if cleaned_text: # 빈 문자열이 아닌 경우만 추가
            filtered_texts.append(cleaned_text)
    # 길이가 1 이상인 값만 추출
    filtered_texts = [text for text in filtered_texts if len(text) > 1]
```


중간프로젝트 | 구현 코드

담당 : 윤수영

신분증 글자 인식 4 : django 모델 구현 및 vue 연결

- Django에서 모델을 생성하고, Vue와 연결하여 데이터가 정상 출력되는지 확인
- 데이터베이스 설계 및 구현, UI 디자인 등을 포함하여 전체 시스템을 통합

The image displays three sequential screenshots of a web application's membership registration process.

Left Screenshot: Registration Form
 Title: 회원가입 (Membership Registration)
 Fields and Buttons:
 - 아이디* (ID): Input field with placeholder '아이디를 입력하세요' and a '중복확인' (Check Duplicate) button.
 - 비밀번호* (Password): Input field with placeholder '비밀번호를 입력하세요'.
 - 비밀번호 확인* (Confirm Password): Input field with placeholder '비밀번호를 한번 더 입력하세요'.
 - 닉네임* (Nickname): Input field with placeholder '닉네임을 입력하세요'.
 - 이메일* (Email): Input field with placeholder '예: ict@market.com' and an '인증' (Verify) button.
 - 휴대폰* (Mobile Phone): Input field with placeholder '숫자만 입력해주세요' (Enter only numbers).
 - 주소* (Address): Input field with placeholder '주소를 입력하세요' and a '주소 검색' (Search Address) button.
 - Bottom: A button '신분증 등록하기' (Register ID Card) and a large orange '가입하기' (Sign Up) button.

Middle Screenshot: File Upload Dialog
 A modal dialog for uploading a profile picture. It shows a preview of a selected image (1.jpg) and a file selection interface with a file type filter set to '사용자 지정 파일 (*.xhtm*.tif*)'. Buttons include '열기(O)' (Open), '취소' (Cancel), and '확인' (Confirm).

Right Screenshot: Confirmation Screen
 Title: 회원가입 (Membership Registration)
 Fields and Buttons:
 - 아이디* (ID): Input field with placeholder '아이디를 입력하세요' and a '중복확인' (Check Duplicate) button.
 - 비밀번호* (Password): Input field with placeholder '비밀번호를 입력하세요'.
 - 비밀번호 확인* (Confirm Password): Input field with placeholder '비밀번호를 한번 더 입력하세요'.
 - 닉네임* (Nickname): Input field with placeholder '닉네임을 입력하세요'.
 - 이메일* (Email): Input field with placeholder '예: ict@market.com' and an '인증' (Verify) button.
 - 휴대폰* (Mobile Phone): Input field with placeholder '숫자만 입력해주세요' (Enter only numbers).
 - 주소* (Address): Input field with placeholder '주소를 입력하세요' and a '주소 검색' (Search Address) button.
 - Bottom: A button '신분증 등록하기' (Register ID Card) and a large orange '가입하기' (Sign Up) button.

중간프로젝트 | 구현 코드

담당 : 윤수영

트리거 - 장바구니 상품 추가, 제거

- 장바구니 테이블에 새로운 상품이 입력되면 상품의 선호도가 1씩 증가하는 트리거
- 반대로 장바구니 테이블에 상품이 삭제되면 상품의 선호도가 1씩 감소하는 트리거

```
CREATE OR REPLACE TRIGGER trg_cart_insert
AFTER INSERT ON Cart
FOR EACH ROW
BEGIN
    UPDATE Products
    SET products_hit = products_hit + 1
    WHERE Products_num = :NEW.Cart_Products_num;
END;
/
```

```
CREATE OR REPLACE TRIGGER trg_cart_delete
AFTER DELETE ON Cart
FOR EACH ROW
BEGIN
    UPDATE Products
    SET products_hit = products_hit - 1
    WHERE Products_num = :OLD.Cart_Products_num;
END;
/
```

중간프로젝트 | 구현 코드

담당 : 윤수영

트리거 - 결제 성공

- 결제를 하면 상품의 선호값이 1씩 증가되고, 상품의 재고는 구매한 수량만큼 감소하는 트리거

```
CREATE OR REPLACE TRIGGER trg_payment_insert
AFTER INSERT ON Payment
FOR EACH ROW
BEGIN
    UPDATE Products
    SET
        products_hit = products_hit + 1,
        products_stock = products_stock - (SELECT Cart_amount FROM Cart WHERE Cart_num
        = (SELECT Orders_Cart_num FROM Orders WHERE Orders_num=:NEW.Payment_Orders_Num))
    WHERE Products_num = (SELECT Cart_Products_num FROM Cart WHERE Cart_num = (SELECT
        Orders_Cart_num FROM Orders WHERE Orders_num = :NEW.Payment_Orders_Num));
END;
/
```

- 결제를 취소하면 상품의 선호값이 1씩 감소되고, 상품의 재고는 결제취소한 수량만큼 증가하는 트리거

```
CREATE OR REPLACE TRIGGER trg_payment_delete
AFTER DELETE ON Payment
FOR EACH ROW
BEGIN
    UPDATE Products
    SET
        products_hit = products_hit - 1,
        products_stock = products_stock + (SELECT Cart_amount FROM Cart WHERE Cart_num
        = (SELECT Orders_Cart_num FROM Orders WHERE Orders_num:OLD.Payment_Orders_Num))
    WHERE Products_num = (SELECT Cart_Products_num FROM Cart WHERE Cart_num = (SELECT
        Orders_Cart_num FROM Orders WHERE Orders_num = :OLD.Payment_Orders_Num));
END;
/
```


중간프로젝트 | 구현 코드

담당 : 윤수영

주문관리

- My batis를 사용하여 주문관리 CRUD 구현
- path에서 주문번호를 매개변수로 주문배송상세를 불러옵니다.

주문관리

전체 주문완료 구매확정 반품요청 반품완료 취소요청 취소완료

배송관리

전체 주문완료 배송준비중 배송중 배송완료

주문번호	주문일자	배송상태	제품명	제품가격	제품수량	고객 ID
12345678	2024-07-08	9	productA		5	customer1
87654321	2024-07-08	1	productB		3	customer2
98765432	2024-07-08	2	productC		2	customer3
23456789	2024-07-08	8	productR		1	customer18
34567890	2024-07-08	9	productS		4	customer19

```
@RequestMapping("/list")
public List<OrderVO> orderlist() {
    return orderService.orderlist();
}

@GetMapping("/detail/{id}")
public OrderVO orderdetail(@PathVariable String id) {
    int vid = Integer.parseInt(id);
    return orderService.orderdetail(vid);
}

@GetMapping("/orderlist/{customerid}")
public List<OrderVO> customerorderlist(@PathVariable String customerid) {
    return orderService.customerorderlist(customerid);
}

@PostMapping("/{id}/update")
public OrderVO upshipping(@PathVariable String id) {
    int vid = Integer.parseInt(id);
    orderService.upshipping(vid);
    return orderService.orderdetail(vid);
}

@PostMapping("/{id}/cancel")
public OrderVO ordercancel(@PathVariable String id) {
    int vid = Integer.parseInt(id);
    orderService.ordercancel(vid);
    return orderService.orderdetail(vid);
}
```


중간프로젝트 | 구현 코드

담당 : 윤수영

주문관리

- 배송 상태값을 업데이트하여 구매완료, 배송상태, 취소여부를 구분

Home Login Sign Up

배송관리상세

주문배송정보

주문번호	고객 ID	제품명	제품수량	제품가격	주문일	배송정보	배송상태변경
12345678	customer1	productA	5	5	2024-07-08	취소완료	배송상태변경

고객정보

번호	아이디	이름	생년월일	연락처	이메일	결제수단	배송지	비고
1	id_1	윤수영	1997-03-07	010-1234-1234	mail@mail.com	1111-2222-3333-4444	경기도	고객정보상세

Second ICT Project 2024 developed by Yun Sooyoung, Kim Geonwoo, Park Seongho, Lee Seunghee, Lee Jiyoung, Jung Joonyoung, Heo Hojun, Song Jimi

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
  PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
  "https://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="kr.co.ict.teamproject.dao.OrderDAO">
  <!-- 전체 목록 조회 -->
  <select id="orderlist" resultType="ordervo">
    SELECT id, customerid, product, quantity, order_date, processing FROM orderboard
  </select>
  <!-- 선택 목록 조회 => 검색 -->
  <select id="orderdetail" resultType="ordervo" parameterType="int">
    SELECT id, customerid, product, quantity, order_date, processing
    FROM orderboard
    WHERE id=#{id}
  </select>
  <select id="customerorderlist" resultType="ordervo" parameterType="string">
    SELECT id, product, quantity, order_date, processing FROM orderboard WHERE customerid=#{customerid}
  </select>
  <!-- 배송상태 수정
  0: 결제완료 1: 배송중 2: 배송완료 3: 구매확정 8: 취소신청 9: 취소완료 -->
  <update id="upshipping" parameterType="int">
    UPDATE orderboard SET processing=processing+1 WHERE id=#{id}
  </update>
  <!-- 주문 상태 수정 -->
  <update id="ordercancel" parameterType="int">
    UPDATE orderboard SET processing=9 WHERE id=#{id}
  </update>
  <!-- 주문 삭제 없음 -->
</mapper>
```

중간프로젝트 | 구현 코드

담당 : 윤수영

OCR 구현 및 테스트

- Google Vision API를 사용하여 이미지 내 글자를 인식하고,
- 인식한 글자 중 이름, 생년월일, 성별을 반환하는 django 모델과 views 코드

회원가입

필수입력사항

아이디* 아이디를 입력하세요 중복확인

비밀번호* 비밀번호를 입력하세요

비밀번호 확인* 비밀번호를 한번 더 입력하세요!

닉네임* 닉네임을 입력하세요

이메일* 예: ict@market.com 인증

휴대폰* 숫자만 입력해주세요

주소* 주소를 입력하세요 주소 검색

계정지에 따라 상품 정보가 달라질 수 있습니다.

□ 신분증 등록하기

이름	윤수영
생년월일	970307
성별	2

가입하기

```
class Project2Model:
    def detect_texts(self, imgName):
        client = vision.ImageAnnotatorClient()
        with io.open(imgName, 'rb') as image_file:
            content = image_file.read()
            image = types.Image(content=content)

        # 텍스트 감지
        response = client.text_detection(image=image)
        texts = response.text_annotations
        print("=====모델에서 텍스트 감지 완료=====")
        # 불필요한 텍스트 패턴
        excluded_patterns = [
            '주민등록증', '2종보통', '자동차운전면허증',
            '운전면허증', '중', '보통', '자동차', '운전', '면허증'
        ]
        filtered_texts = []
        for text in texts:
            cleaned_text = re.sub('[^가-힣ㄱ-ㅎㅏ-ㅣ|0-9]', ' ', text.description)
            cleaned_text = re.sub(r'\d+중', '', cleaned_text)
            for pattern in excluded_patterns:
                cleaned_text = re.sub(pattern, '', cleaned_text)
            cleaned_text = cleaned_text.replace(' ', '') # 모든 공백 제거
            if cleaned_text: # 빈 문자열이 아닌 경우만 추가
                filtered_texts.append(cleaned_text)
        # 길이가 1 이상인 값만 추출
        filtered_texts = [text for text in filtered_texts if len(text) > 1]
        print(filtered_texts)
        return filtered_texts
```

```
@csrf_exempt
def insert_img(request):
    if request.method == 'POST' and 'file1' in request.FILES:
        file = request.FILES['file1']
        file_name = 'file'
        file_path = os.path.join(UPLOAD_DIR, file_name)
        try:
            with open(file_path, 'wb+') as destination:
                for chunk in file.chunks():
                    destination.write(chunk)
        except Exception as e:
            return JsonResponse({"error": f"파일 저장 중 오류 발생: {str(e)}"}, status=400)
        # 이미지 처리
        myModel = Project2Model()
        try:
            texts = myModel.detect_texts(file_path)
            print(f"Detected texts => {texts}")
        except Exception as e:
            return JsonResponse({"error": f"텍스트 감지 중 오류 발생: {str(e)}"}, status=400)
        if len(texts) < 3:
            return JsonResponse({"error": "텍스트 감지 실패"}, status=400)
        try:
            if '경신기간' in texts[0]:
                # 운전면허증
            else:
                # 주민등록증
                name = texts[1]
                birth = texts[2][:6]
                gender = texts[2][6]
                resJson = {"name": name, "birth": birth, "gender": gender}
            return JsonResponse(resJson)
        except IndexError:
            return JsonResponse({"error": "텍스트 감지 실패"}, status=400)
    else:
        return JsonResponse({"error": "파일 업로드 실패"}, status=400)
```


중간프로젝트 | 구현 코드

담당 : 윤수영

추천 알고리즘 구현 및 테스트

- kaggle에서 수집한 데이터를 여러 알고리즘을 사용하여 예측 시도함.
- 랜덤포레스트(0.624) > 결정트리(0.626) > 결정트리 앙상블(0.590) > 로지스틱회귀(0.589)

```
order_products = pd.read_csv(f'{path}/order_products123.csv')
order_products.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 33819106 entries, 0 to 33819105
Data columns (total 16 columns):
 #   Column                Dtype
---  -
 0   Unnamed: 0            int64
 1   order_id              int64
 2   product_id           int64
 3   add_to_cart_order    int64
 4   reordered            int64
 5   user_id              int64
 6   eval_set             object
 7   order_number         int64
 8   order_dow            int64
 9   order_hour_of_day    int64
10   days_since_prior_order float64
11   product_name         object
12   aisle_id             int64
13   department_id        int64
14   aisle               object
15   department           object
dtypes: float64(1), int64(11), object(4)
memory usage: 4.0+ GB
```

```
# 사용자 소비 패턴을 기반으로 예측할 특징과 레이블 선택
features = test_df[['product_id', 'order_dow', 'order_hour_of_day']]
labels = test_df['reordered']
```

```
# 결측값 처리 : 없음
features = features.fillna(0)
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, accuracy_score
from tqdm import tqdm
import pickle
import numpy as np
import pandas as pd
```

```
# 데이터 분할
X_train, X_test, y_train, y_test = train_test_split(features, labels, test_size=0.2, random_state=42)
```

```
# 특징 스케일링
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
# 'prior'인 데이터는 테스트용 데이터프레임으로 나누기
test_df = order_products.loc[order_products['eval_set'] == 'prior']

# 'train'인 데이터는 검증용 데이터프레임으로 나누기
validation_df = order_products.loc[order_products['eval_set'] == 'train']

# 각 데이터프레임의 크기 확인
print(f"테스트용 데이터프레임 크기: {len(test_df)}")
print(f"검증용 데이터프레임 크기: {len(validation_df)}")
```

```
테스트용 데이터프레임 크기: 32434489
검증용 데이터프레임 크기: 1384617
```

```
# Gradient Boosting Machine (GBM)

# 결정 트리를 연속적으로 훈련하여 예측 성능을 향상시키는 앙상블 기법입니다.
from sklearn.ensemble import GradientBoostingClassifier

gbm = GradientBoostingClassifier()
gbm.fit(X_train_scaled, y_train)
y_pred = gbm.predict(X_test_scaled)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.5900296258704854
```

```
# 로지스틱 회귀 (Logistic Regression)
# 이진 분류 문제에서 많이 사용되며, 선형 모델을 기반으로 합니다 (초)
from sklearn.linear_model import LogisticRegression

logistic_regression = LogisticRegression()
logistic_regression.fit(X_train_scaled, y_train)
y_pred = logistic_regression.predict(X_test_scaled)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.5897270159018995
```

```
# 결정 트리 (Decision Tree)

# 데이터의 특징을 기준으로 트리를 생성하여 예측합니다. (분)
from sklearn.tree import DecisionTreeClassifier

decision_tree = DecisionTreeClassifier()
decision_tree.fit(X_train_scaled, y_train)
y_pred = decision_tree.predict(X_test_scaled)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.6266687097592717
```

```
# 랜덤 포레스트 (Random Forest)

# 여러 개의 결정 트리를 앙상블하여 예측 성능을 향상시킵니다. (2시간)
from sklearn.ensemble import RandomForestClassifier

random_forest = RandomForestClassifier()
random_forest.fit(X_train_scaled, y_train)
y_pred = random_forest.predict(X_test_scaled)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.624677735675819
```


중간프로젝트 | 구현 코드

담당 : 윤수영

추천 알고리즘 구현 및 테스트

- KNN을 사용한 추천 알고리즘 학습 및 모델 평가 : 정확도 0.474

```
# 데이터 분할
X_train, X_test, y_train, y_test = train_test_split(features, labels, test_size=0.2, random_state=42)

# 특징 스케일링
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 분류기 모델 생성
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled[:-1], y_train[:-1]) # 일부 데이터 제외하고 훈련

for epoch in tqdm(range(1), desc="Training KNN model"):
    knn.fit(X_train_scaled, y_train)

# 모델과 스케일러 저장
with open('knn_model2.pkl', 'wb') as model_file:
    pickle.dump(knn, model_file)
with open('scaler2.pkl', 'wb') as scaler_file:
    pickle.dump(scaler, scaler_file)

# 예측 해보기
# 새로운 데이터 x에 대한 y값 예측
predict_res = knn.predict(X_train_scaled[-1].reshape(1, -1))

# 최종 결과
print("최종결과 :", y_train[predict_res[0]])
```

```
# 모델과 스케일러 불러오기
with open('/content/drive/MyDrive/Colab Notebooks/ictStudyA/model/knn_model.pkl', 'rb') as model_file:
    loaded_knn = pickle.load(model_file)
with open('/content/drive/MyDrive/Colab Notebooks/ictStudyA/model/scaler.pkl', 'rb') as scaler_file:
    loaded_scaler = pickle.load(scaler_file)

# 불러온 모델로 예측
X_test_scaled_loaded = loaded_scaler.transform(X_test_scaled)
y_pred_loaded = loaded_knn.predict(X_test_scaled_loaded)

# 불러온 모델 평가
print("불러온 모델 평가")
print(classification_report(y_test, y_pred_loaded))
print('Accuracy:', accuracy_score(y_test, y_pred_loaded))
```

불러온 모델 평가

	precision	recall	f1-score	support
0	0.41	0.62	0.49	2661399
1	0.59	0.37	0.46	3825499
accuracy			0.47	6486898
macro avg	0.50	0.50	0.47	6486898
weighted avg	0.51	0.47	0.47	6486898

Accuracy: 0.47493886908658034

중간프로젝트 | 구현 코드

담당 : 윤수영

추천 알고리즘 구현 및 테스트

- 추천 알고리즘에 사용하는 Surprise 라이브러리를 사용하여 모델 학습 및 평가 : 정확도 0.640
- 최고 정확도가 0.64로 정확도를 높이기 위해 특성 컬럼을 바꿔보거나 모델 학습을 딥러닝으로 바꿔보려고 함

```
# Surprise 라이브러리의 Reader 객체를 사용하여 데이터 로드
reader = Reader(rating_scale=(1, 5))

# user_id, product_id, reordered 열을 사용하여 Dataset 객체 생성
data = Dataset.load_from_df(order_products[['user_id', 'product_id', 'reordered']], reader)

# 학습 데이터와 테스트 데이터로 분리
trainset, testset = surprise_train_test_split(data, test_size=0.2)

# SVD 모델 초기화 및 학습
model = SVD()
model.fit(trainset)

# 예측 수행
predictions = model.test(testset)

# 정확도 평가
accuracy.rmse(predictions)
```

```
RMSE: 0.6400
0.6400035402509369
```


감사합니다

윤수영 포트폴리오

연락처 : 010-2175-3797

이메일 : ysy_itc@naver.com